Politecnico di Torino

VANDAL

Robot Learning

Lecture 3. Motor Control

Lecture 3a:

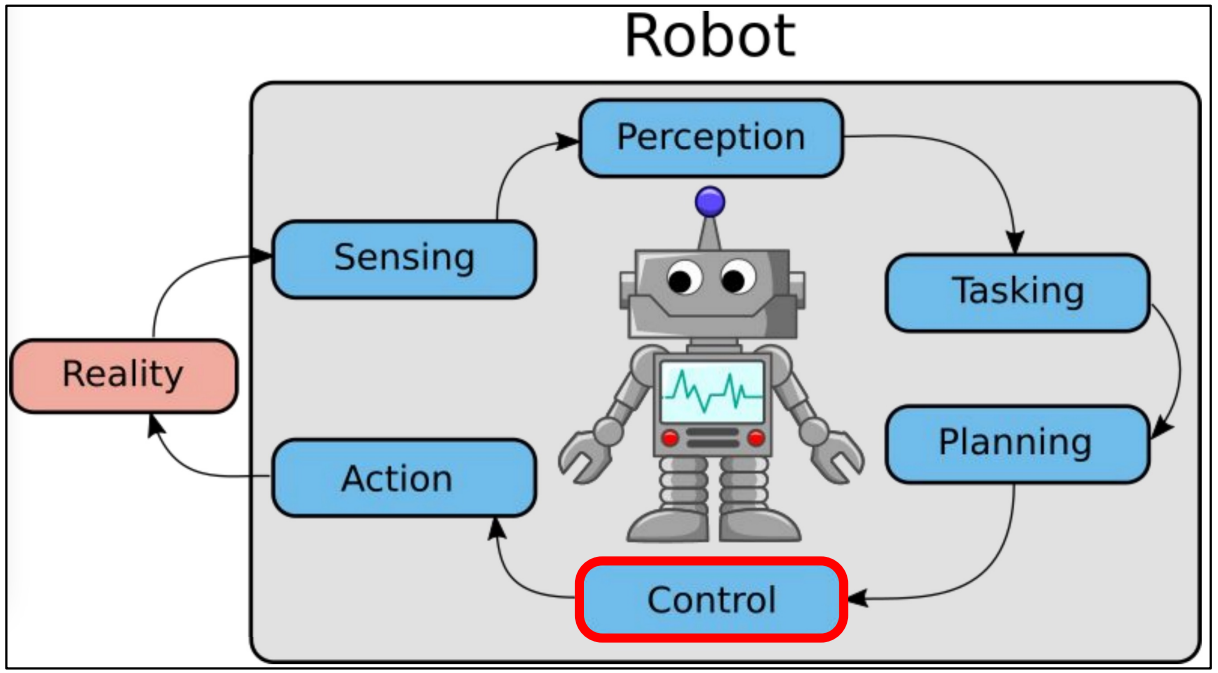
Stability and Regulation

Slides partially adapted from Alessandro de Luca (Sapienza)

Robot Control

From "The intelligent connection of perception to action", J. M. Bradley, MIT AI Lab, 1986

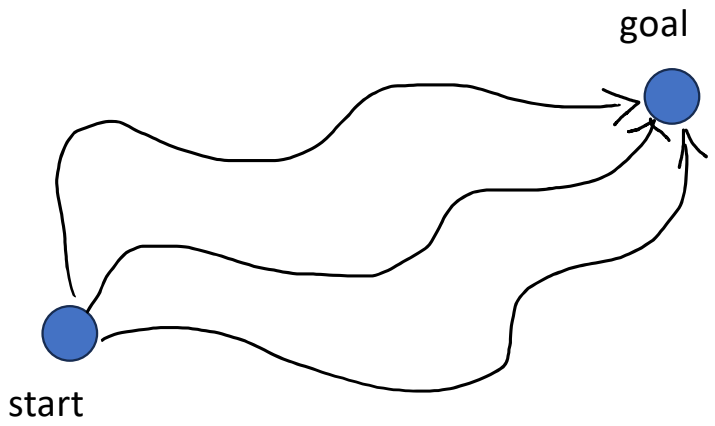
"Robotics is the intelligent connection of the perception to action"



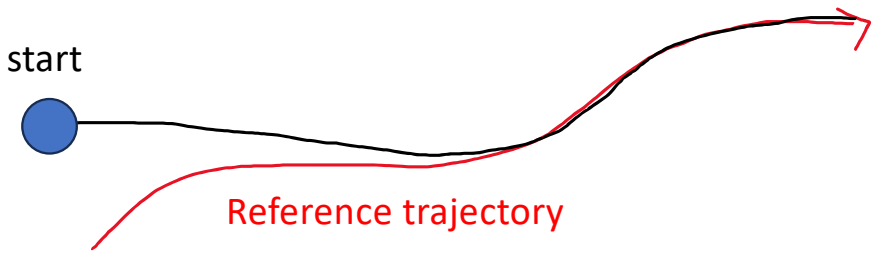
Motion control objectives

Motion control is the problem of defining how an autonomous system moves. We can classify control objectives in three main classes:

1. Regulation: the goal is to reach a **desired fixed configuration** (expressed in joint or task space) and keep that configuration. The behavior during the transient is in general not guaranteed



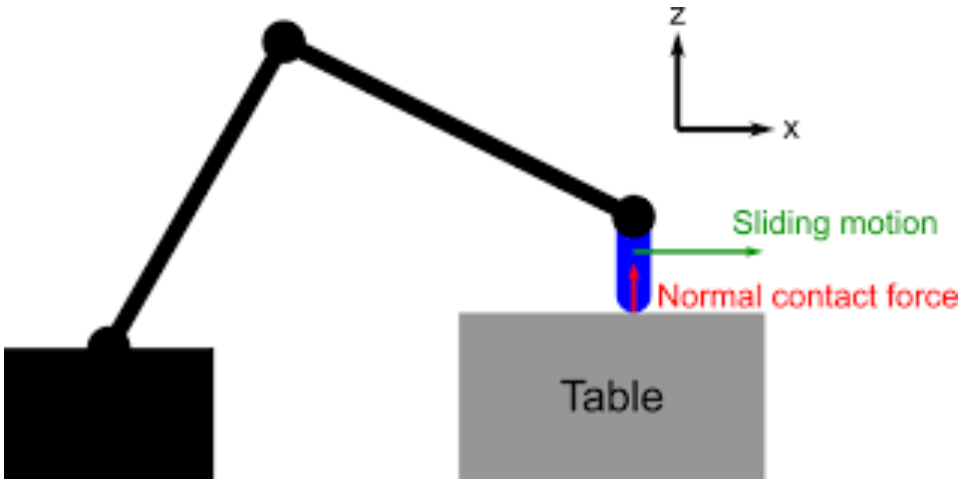
2. Trajectory tracking: the goal is to follow a **desired time-varying reference trajectory** (either in task or joint space).



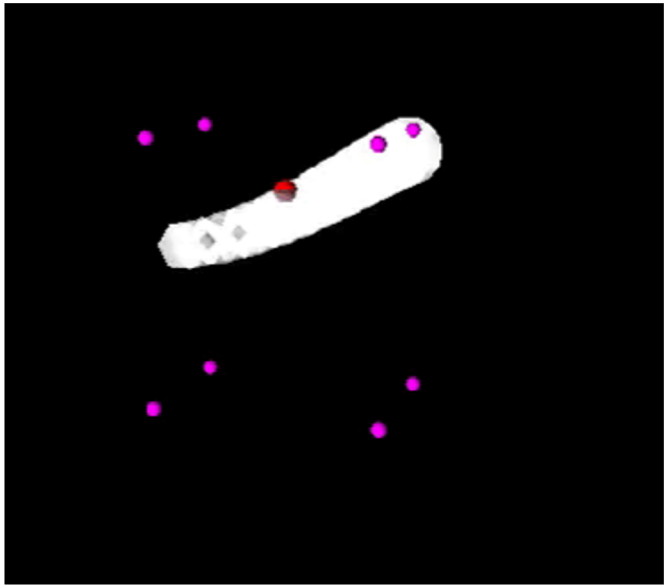
Motion control objectives

Motion control is the problem of defining how an autonomous system moves. We can classify control objectives in three main classes:

3. Contact motion: the goal is for the robot to physically interact with the environment (e.g., pushing on a surface) and thus exchanging forces with the environment.



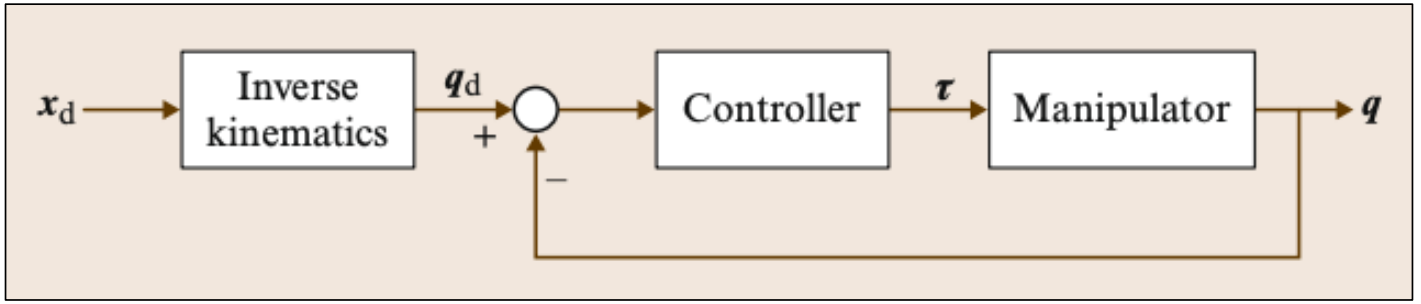
Just an example



Joint space control

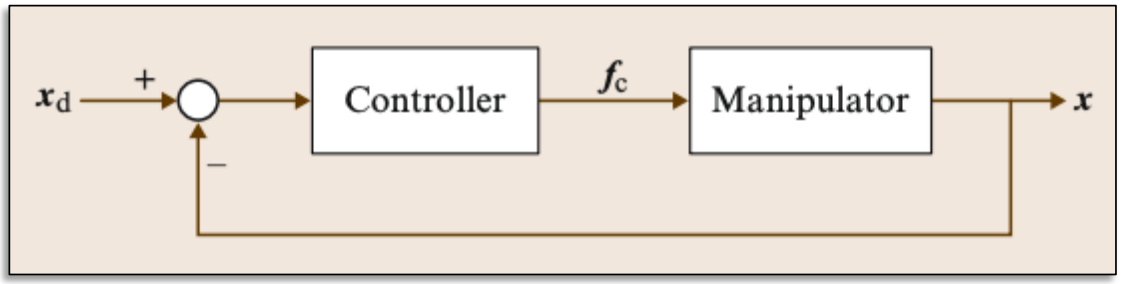
Task specification (end-effector motion and forces) is usually carried out in the operational space, whereas control actions (joint actuator generalized forces) are performed in the joint space.

Since the objective is generally given in task space, it must be converted to the joint space, typically using the **inverse kinematics**.



Task space control

Task space control schemes are usually based on the **dynamics** expressed in the **operational space**.



Joint space dynamics

$$\boldsymbol{\tau} = B(\boldsymbol{q}) + C(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q})$$

Velocity mapping

$$\dot{\boldsymbol{x}} = J(\boldsymbol{q})\dot{\boldsymbol{q}}$$

$$\boldsymbol{f_c} = \Lambda(\boldsymbol{q})\ddot{\boldsymbol{x}} + \Gamma(\boldsymbol{q}, \ddot{\boldsymbol{q}})\,\dot{\boldsymbol{x}} + \boldsymbol{\mu}(\boldsymbol{q})$$

Equilibrium states

Let's consider a non-linear dynamical system in the form

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{h}(\boldsymbol{x})\boldsymbol{u}$$

A state is an **equilibrium** for this system if, without perturbations, **the system stays in that state** once reached (without inputs, or with a suitable external input)

Unforced equilibrium $(\boldsymbol{u} = \boldsymbol{0})$: ⟶ $\boldsymbol{f}(\boldsymbol{x_e}) = \boldsymbol{0}$

Forced equilibrium $(\boldsymbol{u} = \boldsymbol{u}(\boldsymbol{x}))$: ⟶ $\boldsymbol{f}(\boldsymbol{x_e}) + \boldsymbol{h}(\boldsymbol{x_e})\boldsymbol{u}(\boldsymbol{x_e}) = \boldsymbol{0}$

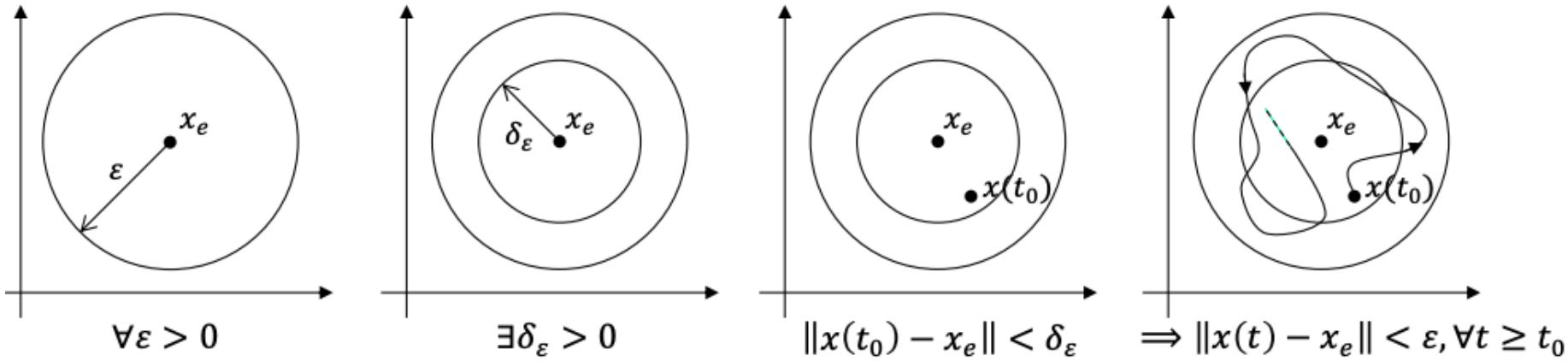
Stability

Let's consider

- a ***nonlinear system*** in the form $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$ (e.g. a closed loop system under feedback control)
- An ***equilibrium*** $\boldsymbol{x}_e$ (i.e., $\boldsymbol{f}(\boldsymbol{x}_e) = \boldsymbol{0}$)

stability of $\boldsymbol{x_e}$

$$\forall \epsilon > 0 : \|x(t_0) - x_e\| < \delta_e \Rightarrow \|x(t) - x_e\| < \epsilon, \forall t \geq t_0$$



Stability

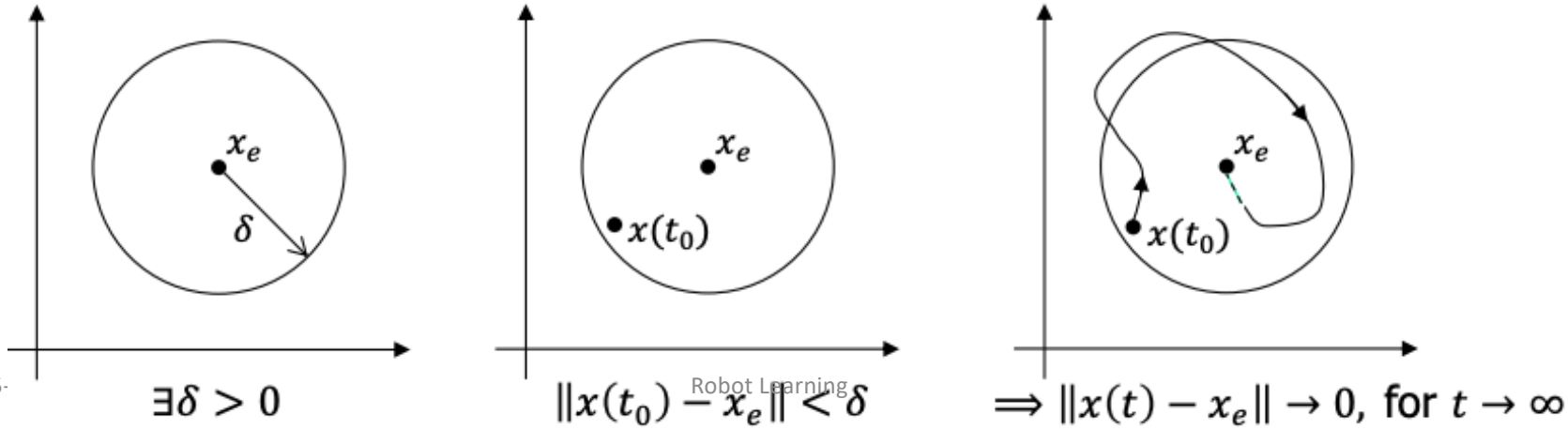
Let's consider

- a ***nonlinear system*** in the form $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$ (e.g. a closed loop system under feedback control)
- An ***equilibrium*** $\boldsymbol{x}_e$ (i.e., $\boldsymbol{f}(\boldsymbol{x}_e) = \boldsymbol{0}$)

Asymptotic stability of $\boldsymbol{x_e}$

x_e stable +

$$\exists \delta > 0 : \|x(t_0) - x_e\| < \delta \Rightarrow \|x(t) - x_e\| \to 0, for\ t \to \infty$$



Stability

Let's consider

- a ***nonlinear system*** in the form $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$ (e.g. a closed loop system under feedback control)
- An ***equilibrium*** $\boldsymbol{x}_e$ (i.e., $\boldsymbol{f}(\boldsymbol{x}_e) = \boldsymbol{0}$)

Exponential stability of $\boldsymbol{x_e}$

$$\exists \delta, c, \lambda > 0 : \|x(t_0) - x_e\| < \delta \Rightarrow \|x(t) - x_e\| \leq c e^{-\lambda(t-t_0)} \|x(t_0) - x_e\|$$

- It allows to estimate the time needed to "approximately" converge: for $c = 1$, when the elapsed time is $t - t_0 = {}^{3}/_{\lambda}$, the initial error is reduced by 50%
- Typically, this is a local property only. Such domain of attraction is hard to be estimated accurately.

How to check stability?

Assessing the **stability of the equilibria** of a nonlinear system is in general non-trivial. We cannot enumerate all possible system trajectories to do it

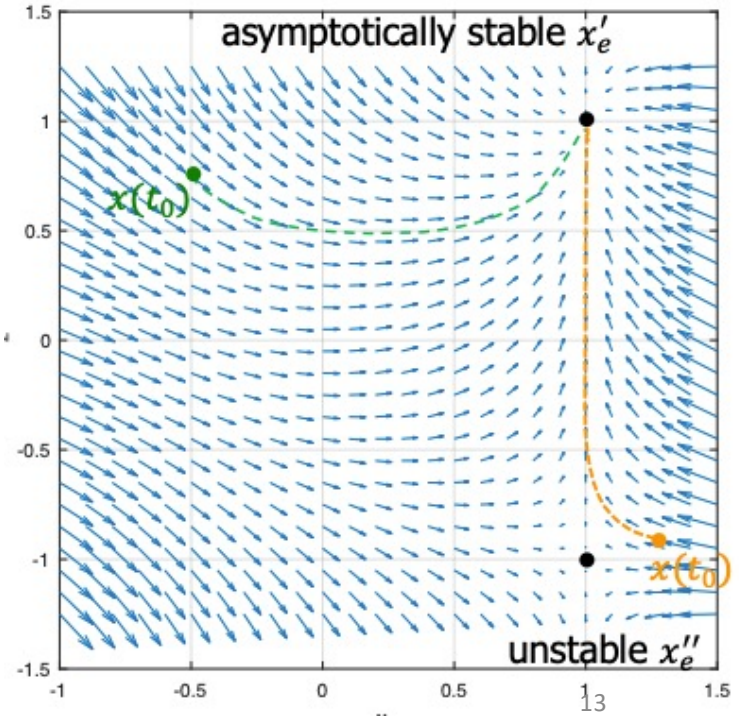
Example

System

$$\begin{cases} \dot{x}_1 = 1 - x_1^3 \\ \dot{x}_2 = x_1 - x_2^2 \end{cases}$$

Equilibria

$$x_e' = (1, 1)$$
$$x_e'' = (1, -1)$$



Lyapunov

Lyapunov candidate

$$V(x)\colon \mathbb{R}^n \to \mathbb{R} \quad \text{s.t.} \quad V(x_e) = 0, V(x) > 0, \forall x \neq x_e$$

- V is a ***positive definite*** function. A typical choice is a **quadratic** function, such as $\frac{1}{2}(x - x_e)^T P(x - x_e)$
- V may be restricted to a local neighbor of x_e, i.e., $\forall\, x : \|x - x_e\| < \delta$

We can provide stability conditions using Lyapunov functions, without the need to enumerate all the possible system trajectories!

Lyapunov

Given a system $\dot{x} = \mathrm{f}(\mathrm{x})$ and a Lyapunov candidate V for the equilibrium x_e, the following results hold.

***Sufficient* condition of stability**

$$\exists V\colon \dot{V}(x) \leq 0 \text{ along the trajectories of } \dot{x} = f(x)$$

***Sufficient* condition of asymptotic stability**

$$\exists V\colon \dot{V}(x) < 0 \text{ along the trajectories of } \dot{x} = f(x)$$

***Sufficient* condition of instability**

$$\exists V\colon \dot{V}(x) > 0 \text{ along the trajectories of } \dot{x} = f(x)$$

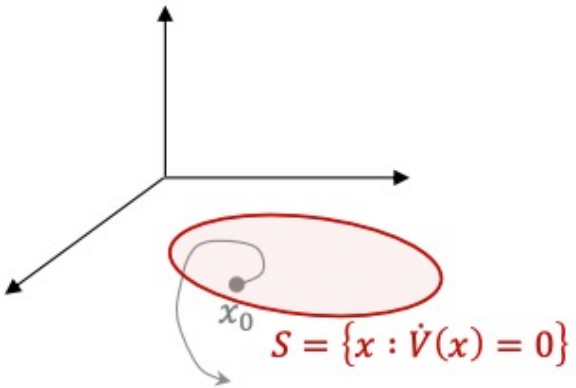
LaSalle Theorem

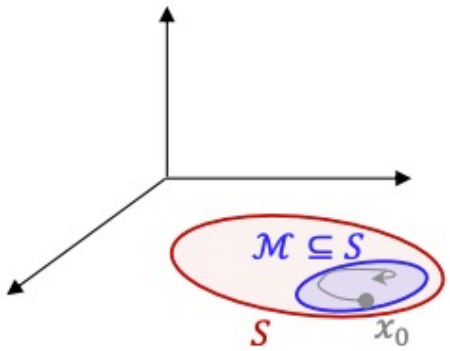
If $\exists V$ candidate : $\dot{V}(x) \leq 0$ along the trajectories of $\dot{x} = f(x)$

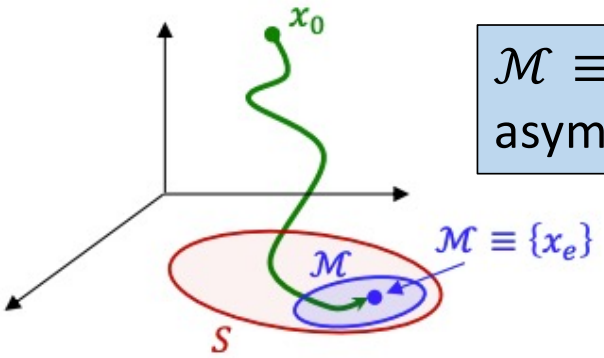
Then system trajectories asymptotically converge to the **largest invariant set**

$\mathcal{M} \subseteq S = \{x \in \mathbb{R}^n : \dot{V}(x) = 0\}$

$\mathcal{M}$ is invariant **if** $\boldsymbol{x(t_0) \in \mathcal{M} \Rightarrow x(t) \in \mathcal{M}, \forall t \geq t_0}$







$\mathcal{M} \equiv x_e$ implies asymptotic stability

Stability of linear systems

$\dot{x} = Ax$

$x_e = 0$ is always an equilibrium state

1. Asymptotic stability
2. **Global** asymptotic stability
3. **Exponential** stability
4. $\sigma(A) \subset \mathbb{C}^-$ (all eigenvalues of A have **negative real part**)
5. $\forall Q > 0$ (positive definite), $\exists! P > 0 : A^T P + PA = -Q$ and the function $\frac{1}{2} x^T P x$ is a **Lyapunov candidate**

Lyapunov equation

ALL EQUIVALENT!

If $x_e = 0$ is an asymptotically stable equilibrium, then it is necessarily the unique equilibrium

Linear approximation

Let $\Delta x = x - x_e$ and let $\dot{\Delta x} = \left.\frac{df}{dx}\right|_{x=x_e} (x - x_e) = A\Delta x$ be the **linear approximation** of $\dot{x} = f(x)$ around the equilibrium x_e

A asymptotically stable ($\sigma(A) \subset \mathbb{C}^-$)

The original nonlinear system is **exponentially** stable at the origin

Note: this is only a **local** result

Regulation Control

PD control

Robot: $B(\boldsymbol{q})\ddot{\boldsymbol{q}} + C(\boldsymbol{q}, \dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{u}$

Goal: asymptotic stabilization (i.e., **regulation**) of the closed-loop equilibrium. Namely, we want to achieve $q = q_d$ with $\dot{q} = 0$

q_d is possibly obtained from kinematic inversion: $q_d = f^{-1}(r_d)$

PD control law

$u = K_P(q_d - q) - K_D\dot{q}$ with $K_P, K_D > 0$ positive definite and symmetric

Example: PD for serial manipulator

In the absence of gravity ($g(q) \equiv 0$), the robot state $(q_d, 0)$ under a PD joint control law is **globally asymptotically stable**.

Proof

Robot: $B(\boldsymbol{q})\ddot{\boldsymbol{q}} + C(\boldsymbol{q}, \dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{u}$

Let's define:

- $e = q_d - q$
- $V = \frac{1}{2}\dot{q}^T B(q)\dot{q} + \frac{1}{2}e^T K_P e \geq 0$

$$V = 0 \Leftrightarrow e = \dot{e} = 0$$

Equal to zero if C is built using Christoffel symbols

$$\dot{V} = \dot{q}^T B\ddot{q} + \frac{1}{2}\dot{q}\dot{B}\dot{q} - e^T K_P \dot{q} = \dot{q}^T\left(u - C\dot{q} + \frac{1}{2}M\dot{q}\right) - e^T K_P \dot{q}$$

$$= \dot{q}^T K_P e - \dot{q}^T K_D \dot{q} - e^T K_P \dot{q} = -\dot{q}^T K_D \dot{q} \leq 0$$

This only proves ***stability***

Example: PD for serial manipulator

We have proven simple stability, but we also know that $\dot{V} = 0 \Leftrightarrow \dot{\boldsymbol{q}} = \mathbf{0}$

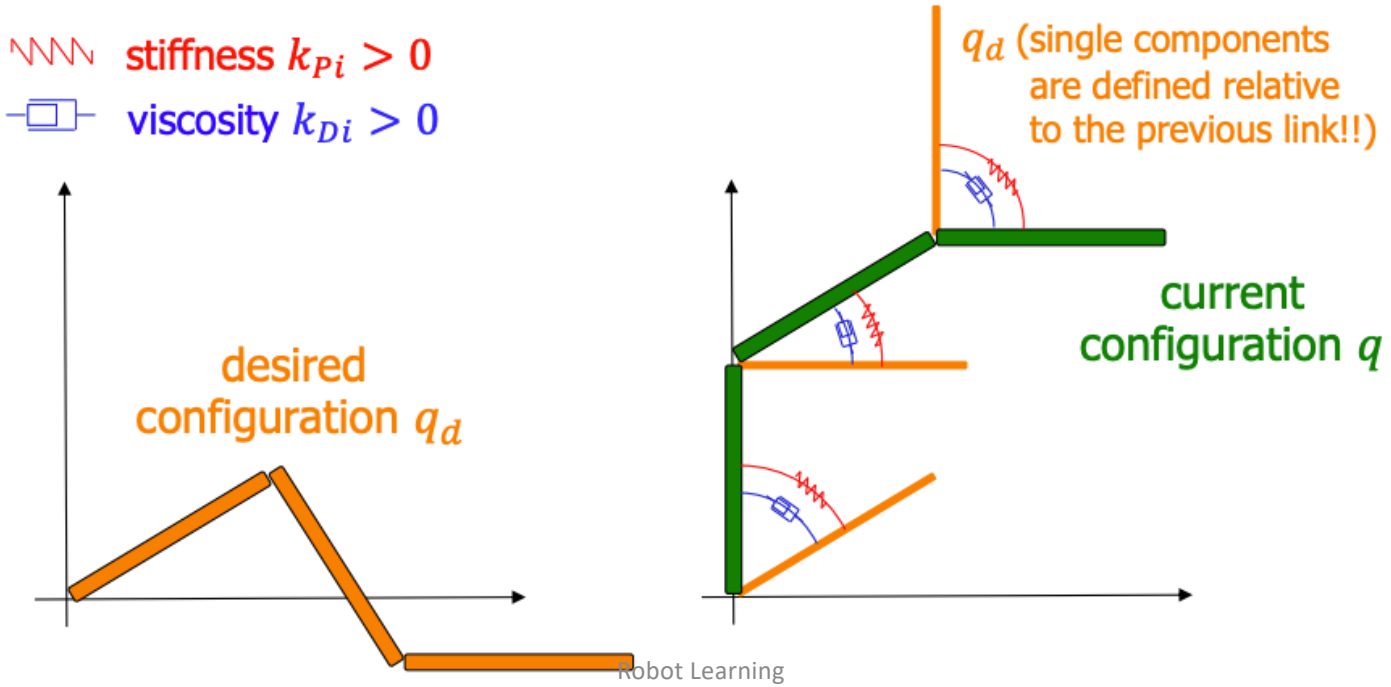
From **LaSalle** theorem, we know that system trajectories converge to the largest **invariant set** of states $\mathcal{M}$ where $\dot{\boldsymbol{q}} = \mathbf{0}$ (that is $\dot{\boldsymbol{q}} = \ddot{\boldsymbol{q}} = \mathbf{0}$)

$$B(\boldsymbol{q})\ddot{\boldsymbol{q}} + C(\boldsymbol{q}, \dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} + \cancel{\boldsymbol{g}(\boldsymbol{q})} = K_P \boldsymbol{e} - K_D(\mathbf{0} - \dot{\boldsymbol{q}}) \xrightarrow{\dot{\boldsymbol{q}} = \mathbf{0}} \ddot{\boldsymbol{q}} = B^{-1}(\boldsymbol{q}) K_P \boldsymbol{e}$$

This means that $\dot{\boldsymbol{q}} = \mathbf{0}, \ddot{\boldsymbol{q}} = \mathbf{0} \Leftrightarrow \boldsymbol{e} = \mathbf{0}$ ⇒ The only invariant state in $\dot{V} = 0$ is given by $q = q_d, \dot{q} = 0$

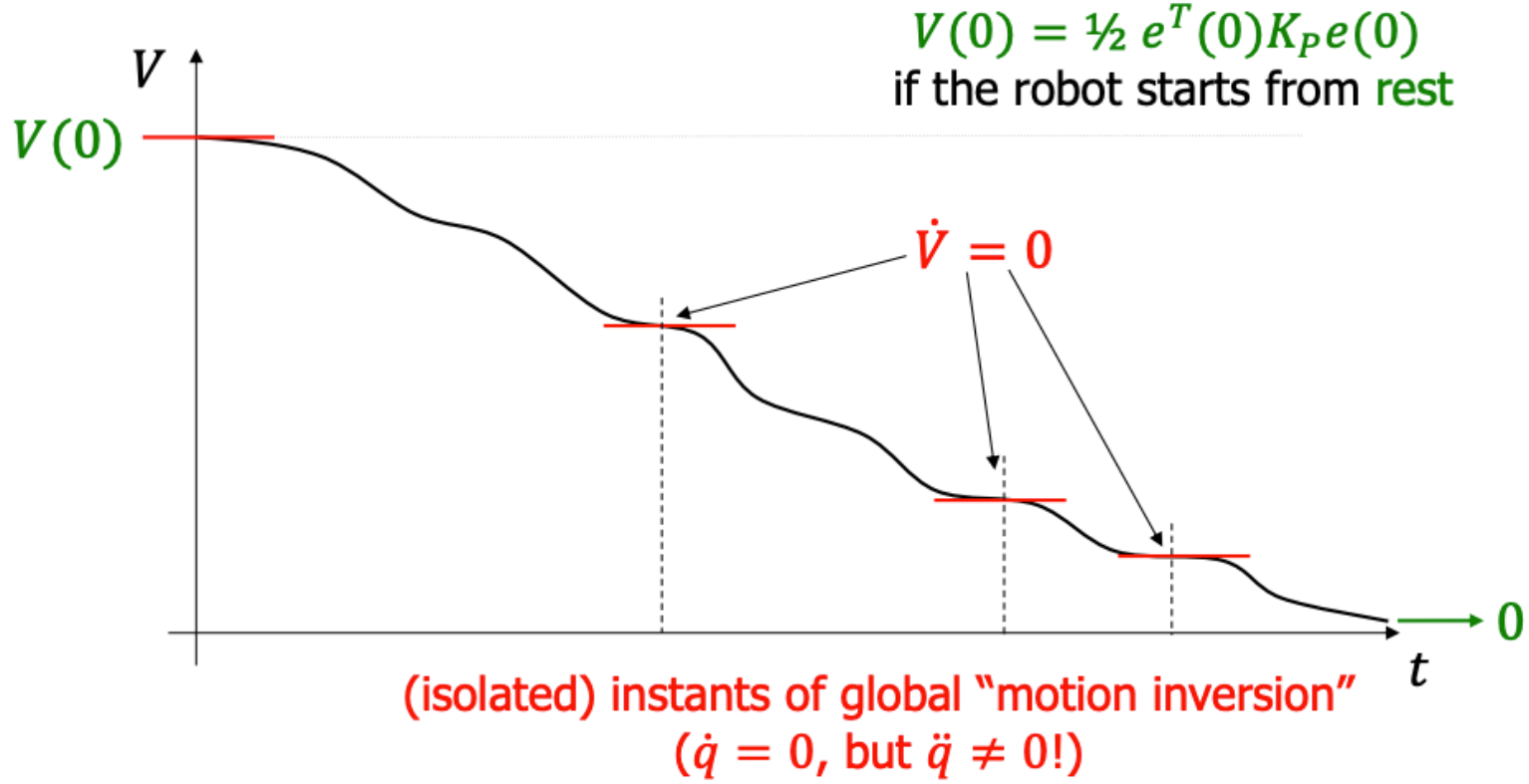
Physical interpretation

For **diagonal** positive definite gain matrices K_P and K_D (thus with positive diagonal elements), such values correspond to stiffness of "virtual" **springs** and viscosity of "virtual" **dampers** placed at the joints



Plot of the Lyapunov function

Time evolution of the Lyapunov candidate



Inclusion of gravity

- In presence of gravity, we could modify the PD controller adding a **gravity compensation term**.

$$\boldsymbol{u} = K_P(\boldsymbol{q}_d - \boldsymbol{q}) - K_D\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q})$$

- However, if gravity is only approximately compensated, i.e.,

$$\boldsymbol{u} = K_P(\boldsymbol{q}_d - \boldsymbol{q}) - K_D\dot{\boldsymbol{q}} + \hat{\boldsymbol{g}}\,(\boldsymbol{q}) \quad \text{with} \quad \hat{\boldsymbol{g}}\,(\boldsymbol{q}) \neq \boldsymbol{g}(\boldsymbol{q})$$

then, it is $\boldsymbol{q} \to \boldsymbol{q}^* \neq \boldsymbol{q}_d, \dot{\boldsymbol{q}} \to 0$ (**steady-state position error**)

- The equilibrium $\boldsymbol{q}^*$ is in general **not unique**, and we have that $\boldsymbol{q}^* \to \boldsymbol{q}_d$ when $K_P \to \infty$

PID control

In comparison to the PD control, the PID adds an **integral control action** that is used to eliminate a **constant error** in the step response at **steady state** (in linear systems)

- For example, in a manipulator we can use it to recover the error due to an absent or incomplete gravity compensation

$$u(t) = K_P(q_d - q) + K_I \int_o^t (q_d - q(\tau))d\tau - K_D\dot{q}(t)$$

- **If** the desired closed-loop equilibrium is asymptotically stable under the PID control, the **integral term at steady state compensates the gravity**

Comments on PID control

- The **choice of the control** gains K_P and K_D affects the evolution of the robot during transients and influences the settling times
  - **Hard to define optimal values**, let alone values that are "optimal" in the whole workspace
  - "full" K_P and K_D gain matrices allow to assign desired eigenvalues to the **linear approximation** of the robot dynamics around the final desired state $(\boldsymbol{q}_d, 0)$
- **Viscous friction** at the joint $-F_v\dot{\boldsymbol{q}}$ acts similarly to the derivative term $-K_D\dot{\boldsymbol{q}}$

Comments on PID control

- The **response times** needed to reach a desired state are not easily predictable in advance
- The **integral term** needs some time to “unload” itself from the error history accumulated during transients (possible to use anti-windup schemes or saturation) (like a capacitor)

Linear Quadratic Regulator

Linear Quadratic Regulator (LQR)

- The linear quadratic regulator is an **optimal control** scheme for stabilizing a **time-invariant linear system** to the **origin**.

Dynamical system

$$x_{t+1} = A\,x_t + Bu_t$$

Quadratic cost function over a finite time-horizon

$$J(X,U) = \sum_{t=0}^{N-1} (x_t^T Q x_t + u_t^T R u_t) + x_N^T Q_f x_N$$

state cost

final state cost

input cost

where $Q, R, Q_f \succ 0$ and symmetric.

This means positive definite, i.e., $x^T Q x > 0\ \forall x \neq 0$

Symmetric matrices

- For any symmetric matrix Q, we can express it in the form

$$Q = V \Lambda V^T$$

a.k.a. eigendecomposition, valid when Q has linearly independent eigenvectors (not necessarily distinct eigenvalues).

where Λ is diagonal and $V^T V = I$

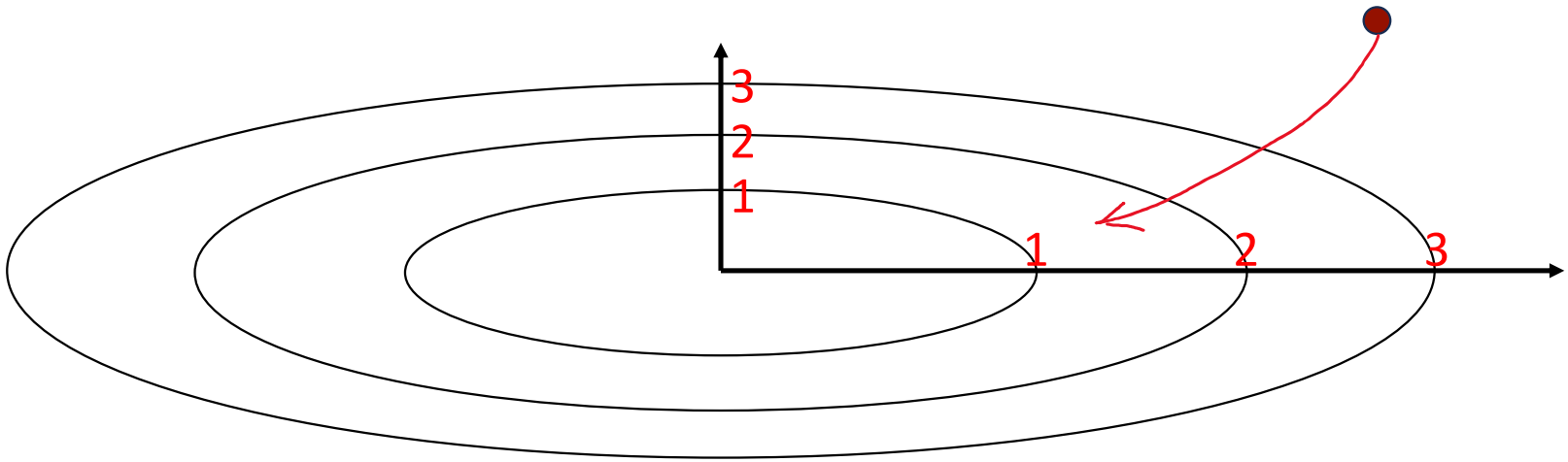
- Therefore, the quadratic form can be thought of as **rotating the vectors via a rotation matrix V**, and rescaling them via Λ

$$x^T Q x = (x^T V) \Lambda (V^T x)$$

Rotated vectors

Symmetric matrices and cost

- So, the cost term $\mathrm{x}_t^{\mathrm{T}} Q x_t$ is effectively **penalizing each coordinate from being far from zero**, but not necessarily in an axis aligned way.
- Q and R can be effectively any positive-definite symmetric matrices, but in practice it is simpler to pick them as **diagonal**.



Non-symmetric matrices

- Let's consider $x^T F\ x$ where F is non symmetric
- We can decompose the matrix in its symmetric and non-symmetric parts

$$F = \frac{F + F^T}{2} + \frac{F - F^T}{2}$$

symmetric ***non-symmetric***

$$\Rightarrow \quad x^T F x = x^T \frac{F+F^T}{2} x + x^T \frac{F-F^T}{2} x = x^T \frac{F+F^T}{2} x + x^T \frac{F}{2} x - x^T \frac{F^T}{2} x$$

$$= x^T \frac{F + F^T}{2} x + x^T \frac{F}{2} x - \left(x^T \frac{F^T}{2} x \right)^T = x^T \frac{F + F^T}{2} x$$

Only the symmetric part matters, so there is no point in using a non-symmetric matrix for Q and R!

LQR derivation

We could write the problem in compact form and try to solve it all at once.

1. Total cost

$$J(U) = x_0^T Q\, x_0 + [x_1^T \dots x_N^T] \underbrace{\begin{bmatrix} Q & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & Q_f \end{bmatrix}}_{\bar{Q}} \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} + U^T \underbrace{\begin{bmatrix} R & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & R \end{bmatrix}}_{\bar{R}} U$$

2. System trajectory

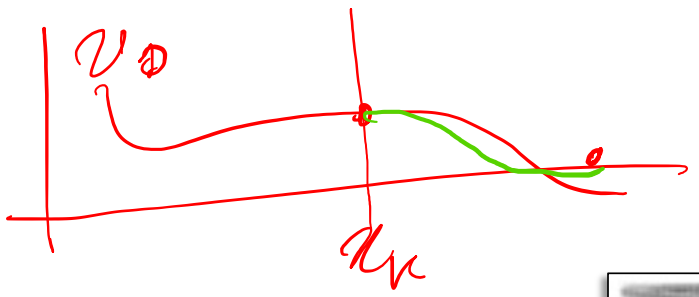
$$\begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} = \underbrace{\begin{bmatrix} B & \cdots & 0 \\ AB & \ddots & \vdots \\ A^{N-1}B & \cdots & B \end{bmatrix}}_{\bar{S}} U + \underbrace{\begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}}_{\bar{T}} x_0$$

It is possible to show (plug 2. in 1. and take the gradient w.r.t. U) that the solution has the form

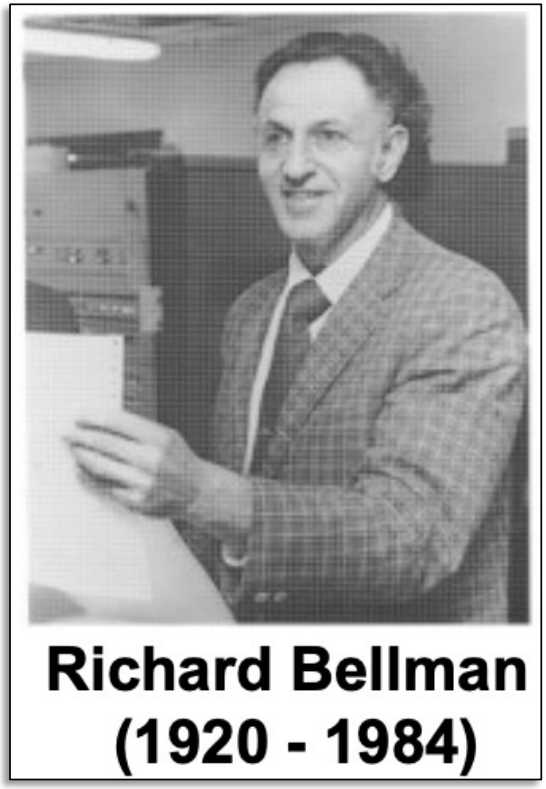
$$U^* = -H^{-1}F^T x_0 \quad \text{where } H = \bar{R} + \bar{S}^T \bar{Q} \bar{S} \text{ and } F = 2\bar{T}^T \bar{Q} \bar{T}$$

Problem: the size of the matrices H and F is proportional to the length of the horizon!

Bellman principle



Given an optimal control sequence $\mathrm{U}^* = [u_0^*, \dots, u_{N-1}^*]$ and the corresponding optimal trajectory $x^*(k)$, the sub-sequence $[u_{t_1}^*, \dots, u_{N-1}^*]$ is still optimal for the horizon $[t_1, N]$ starting from the optimal state $x^*(t_1)$



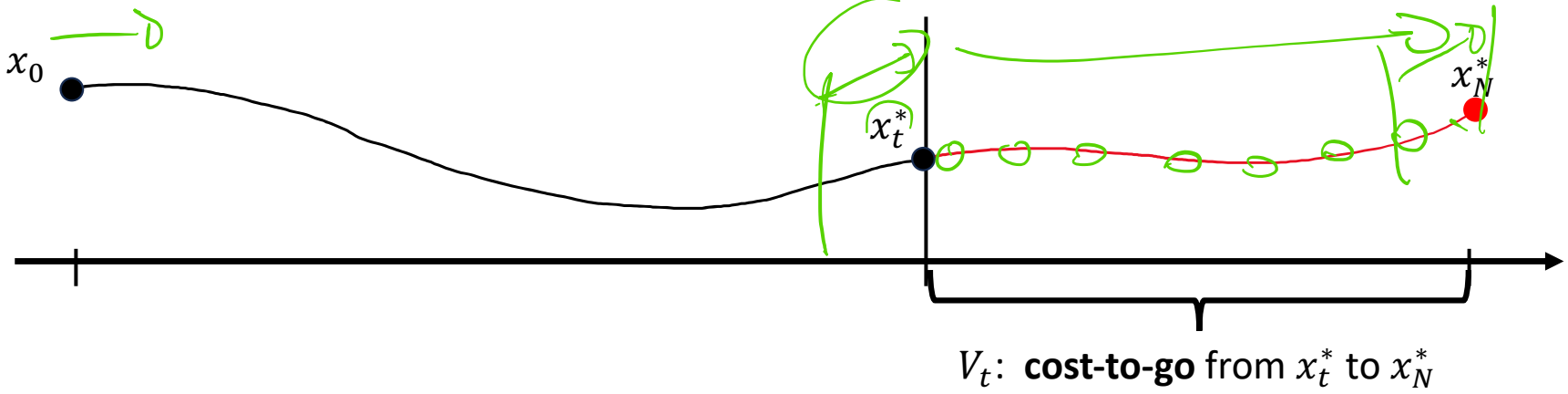
Richard Bellman
(1920 - 1984)

We can use this principle to derive an iterative solution

LQR iterative solution

Let's define the ***cost-to-go***, as the residual of the cost in the interval $[t, N]$ starting from x_t

$$V_t = \min_{u_t,\dots,u_{N-1}} \left[\sum_{\tau=t}^{N-1} (x_\tau^T Q x_\tau + u_\tau^T R u\_\tau) + x_N^T Q_f x_N \right]$$



Idea: start by optimizing the last interval, and then roll-back in time until we reach the start

LQR iterative solution

- The cost-to-go for the last time-step is:

$$V_N(x_N) = \underbrace{x_N^T Q_f x_N}$$

Let's also call this matrix P_N

- Going back to $N-1$ we have:

$$V_{N-1}(x_{N-1}) = \min_{u_{N-1}} x_{N-1}^T Q\, x_{N-1} + u_{N-1}^T R u_{N-1} + V_N(x_N)$$

$$= \min_{u_{N-1}} x_{N-1}^T Q\, x_{N-1} + u_{N-1}^T R u_{N-1} + V_N(Ax_{N-1} + Bu_{N-1})$$

$$= \min_{u_{N-1}} \underbrace{x_{N-1}^T Q\, x_{N-1} + u_{N-1}^T R u_{N-1} + (Ax_{N-1} + Bu_{N-1})^T Q_f (Ax_{N-1} + Bu_{N-1})}_{g_{N-1}(x_{N-1}, u_{N-1})}$$

LQR iterative solution

The optimal u^*_{N-1} is found by setting the gradient of the term $g(x_{N-1}, u_{N-1})$ w.r.t. u_{N-1} equal to zero

$$g_{N-1}(x_{N-1}, u_{N-1}) = x_{N-1}^T Q\, x_{N-1} + u_{N-1}^T R u_{N-1} + (Ax_{N-1} + Bu_{N-1})^T Q_f (Ax_{N-1} + Bu_{N-1})$$

$$\nabla_{u_{N-1}} g_{N-1}(x_{N-1}, u_{N-1}) = 2Ru_{N-1} + 2B^T Q_f (Ax_{N-1} + Bu_{N-1}) = 0$$

$$u^*_{N-1} = -\left(R + B^T Q_f B\right)^{-1} B^T Q_f A x_{N-1} = K x_{N-1}$$

We can solve this single step nicely, and find a **closed-form solution**!

LQR iterative solution

With the optimal input for the step $N-1$, we can also comput the exact cost-to-go V_{N-1}

$$V_{N-1}(x_{N-1}) = \min_{u_{N-1}} x_{N-1}^T Q\, x_{N-1} + u_{N-1}^T R u_{N-1} + (Ax_{N-1} + Bu_{N-1})^T Q_f (Ax_{N-1} + Bu_{N-1})$$

$$= x_{N-1}^T \left[Q - A^T Q_f B \left(R + B^T Q_f B \right)^{-1} B^T Q_f A + A^T Q_f A \right] x_{N-1}$$

$$= x_{N-1}^T P_{N-1} x_{N-1}$$

The cost-to-go has a quadratic expression similar to the one we started with!
This means we can keep iterating back in time using the same procedure.

LQR: iteration

1. Set $P_N = Q_f$

2. For $t = N, \dots, 1$ set
$$P_{t-1} := Q + A^T P_t A - A^T P_t B (R + B^T P_t B)^{-1} B^T P_t A$$
→ Matrix to compute the cost-to-go
$V_{t-1}(x_{t-1}) = x_{t-1}^T P_{t-1} x_{t-1}$

3. For $t = 0, \dots, N-1$ set
$$K_t := -(R + B^T P_{t+1} B)^{-1} B^T P_{t+1} A$$
→ Matrix to compute the optimal control input

4. For $t = 0, \dots, N-1$ the optimal input is given by
$$u_t^* = K_t x_t$$

LQR: infinite horizon

- In the case of an infinite horizon, the LQR control is

$$u = Kx_t$$

where

$$K = -(R + B^T P B)^{-1} B^T P A$$

$$P = Q + A^T P A - A^T P B (B^T P B)^{-1} P A$$

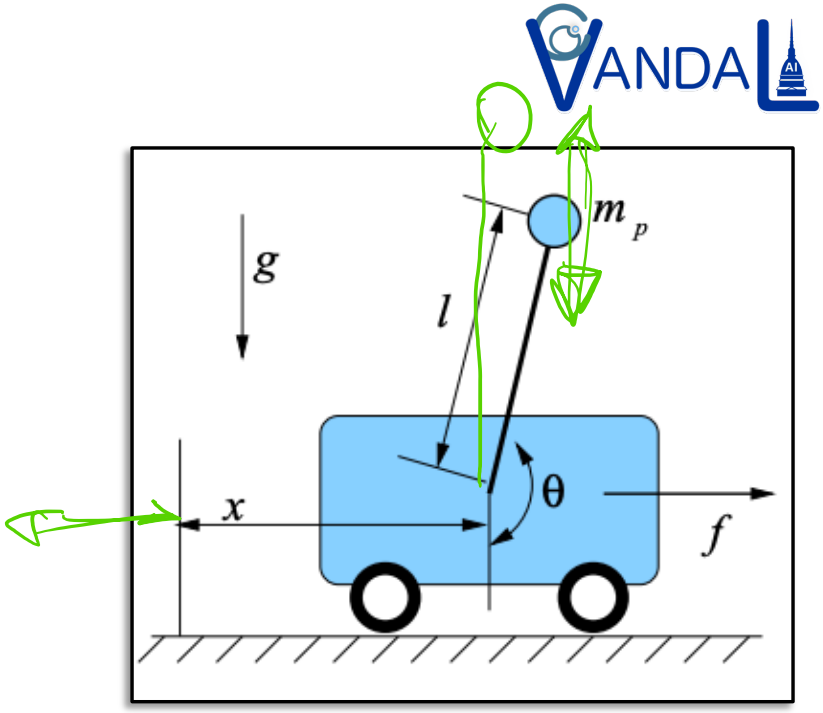
which is called the discrete-time **algebraic Riccati equation** (ARE).

- Note that the matrices K and P are time-invariant
- We don't have a recursion to compute P. We can only solve the ARE

Example: LQR for cart-pole

Configuration: $\boldsymbol{q} = [x, \theta]^T$

Goal: regulate the system to reach the configuration $\boldsymbol{q_d} = [0, \pi]^T$ with zero speed (i.e., $\dot{\boldsymbol{q}}_d = [0, 0]^T$)



Energy:

$$T = \frac{1}{2}(m_c + m_p)\dot{x}^2 + m_p \dot{x}\dot{\theta} l \cos\theta + \frac{1}{2} m_p l^2 \dot{\theta}^2$$

$$U = -\mathrm{m_p g l} \cos\theta$$

Equations of motion (from Lagrangian)

$$(m_c + m_p)\ddot{x} + m_p l \ddot{\theta} \cos\theta - m_p l \dot{\theta}^2 \sin\theta = f$$
$$m_p l \ddot{x} \cos\theta + m_p l^2 \ddot{\theta} + m_p g l \sin\theta = 0$$

$$B(\boldsymbol{q}) = \begin{bmatrix} m_c + m_p & m_p l \cos\theta \\ m_p l \cos\theta & m_p l^2 \end{bmatrix}$$

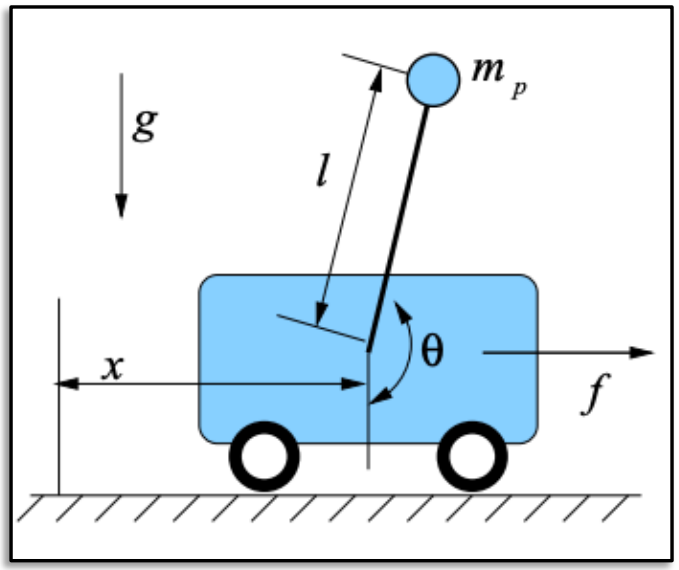
$$C(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \begin{bmatrix} 0 & -m_p l \dot{\theta} \sin\theta \\ 0 & 0 \end{bmatrix} \qquad \boldsymbol{u} = \begin{bmatrix} f \\ 0 \end{bmatrix}$$

$$\boldsymbol{g}(\boldsymbol{q}) = \begin{bmatrix} 0 \\ m_p g l \sin\theta \end{bmatrix}$$

Example: LQR for cart-pole

To apply the LQR to this system, we need to linearize the model around the desired equilibrium, using a Taylor expansion.

$$B(\boldsymbol{q})\ddot{\boldsymbol{q}} + C(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{u}$$



Let's consider the extended state $\boldsymbol{x_e} = [\boldsymbol{q}, \dot{\boldsymbol{q}}]$. We have

$$\dot{\boldsymbol{x}}_{\boldsymbol{e}} = \begin{bmatrix} \dot{\boldsymbol{q}} \\ \ddot{\boldsymbol{q}} \end{bmatrix} = \begin{bmatrix} \dot{\boldsymbol{q}} \\ B^{-1}(\boldsymbol{q})[\boldsymbol{u} - C(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} - \boldsymbol{g}(\boldsymbol{q})] \end{bmatrix} = \boldsymbol{h}(\boldsymbol{x}_e, \boldsymbol{u})$$

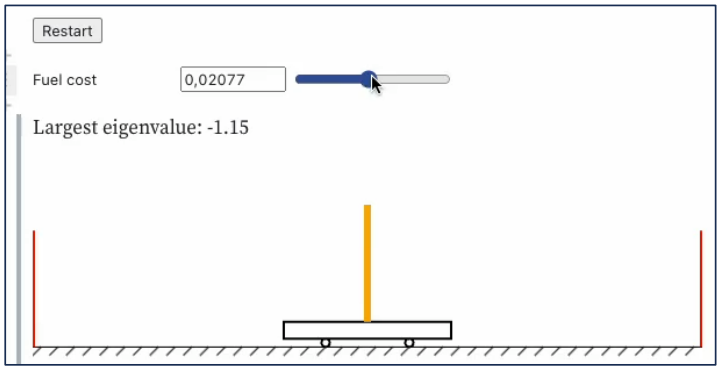
$$\approx \boldsymbol{h}(\boldsymbol{x}_{\boldsymbol{e}}^*, \boldsymbol{u}^*) + (\boldsymbol{x}_{\boldsymbol{e}} - \boldsymbol{x}_{\boldsymbol{e}}^*)\left[\frac{d\boldsymbol{h}}{d\boldsymbol{x}_e}\right]_{\boldsymbol{x_e}=\boldsymbol{x}_{\boldsymbol{e}}^*, \boldsymbol{u}=\boldsymbol{u}^*} + (\boldsymbol{u} - \boldsymbol{u}^*)\left[\frac{d\boldsymbol{h}}{d\boldsymbol{u}}\right]_{\boldsymbol{x}=\boldsymbol{x}^*, \boldsymbol{u}=\boldsymbol{u}^*} = A(\boldsymbol{x_e} - \boldsymbol{x}_{\boldsymbol{e}}^*) + B(\boldsymbol{u} - \boldsymbol{u}^*)$$

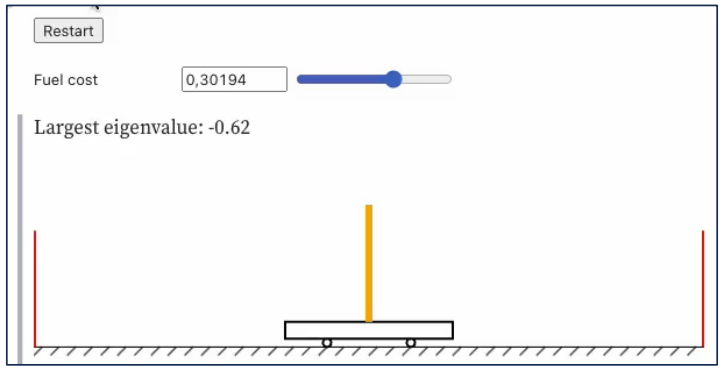
=0, it's an equilibrium

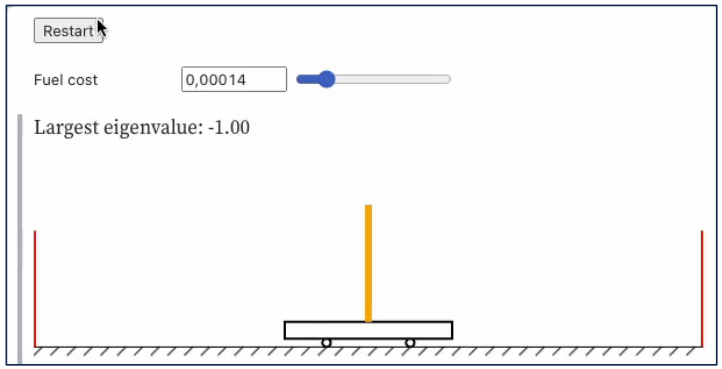
$$A = \begin{bmatrix} 0 & I \\ -B^{-1}\dfrac{d\boldsymbol{g}}{d\boldsymbol{q}} & -B^{-1}C \end{bmatrix}_{\boldsymbol{x_e}=\boldsymbol{x}_{\boldsymbol{e}}^*, \boldsymbol{u}=\boldsymbol{u}^*} \qquad B = \begin{bmatrix} 0 \\ B^{-1} \end{bmatrix}_{\boldsymbol{x}=\boldsymbol{x}^*, \boldsymbol{u}=\boldsymbol{u}^*}$$

Example: LQR for cart-pole

- Let’s try changing the penalty for the inputs (here called fuel cost)

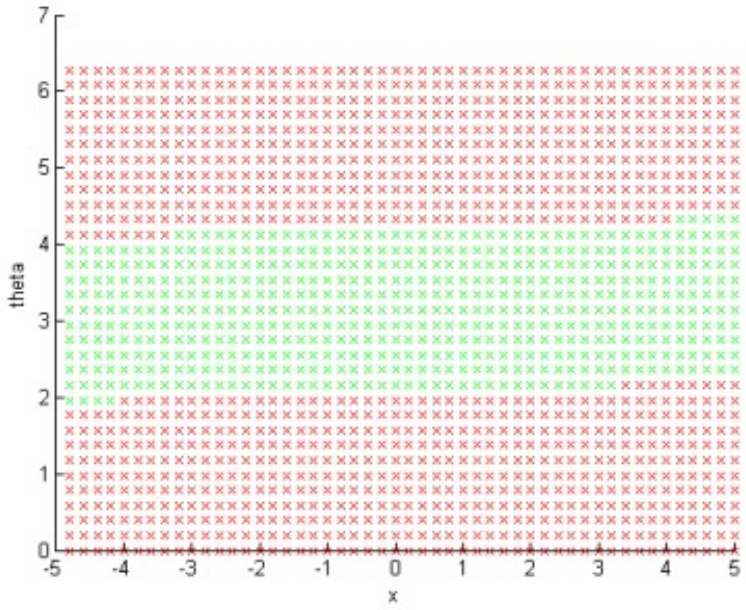






Example: LQR for cart-pole

Results of running LQR for the linear time-invariant system obtained from linear
[0;0;0;0]. The cross-marks correspond to initial states. Green means the contro
at stabilizing from that initial state, red means not.



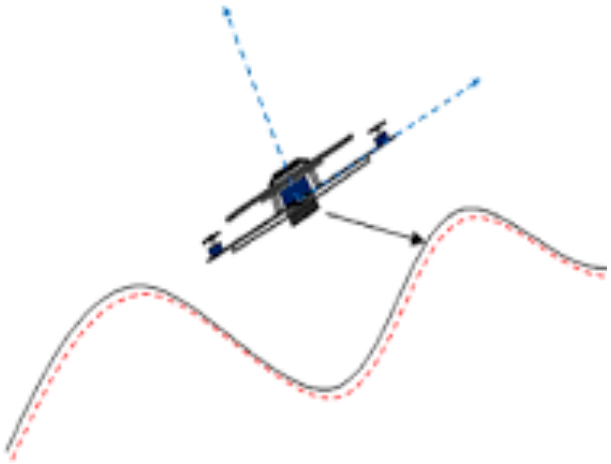
Q = diag([1;1;1;1]); R = 0; [x, theta, xdot, thetadot]

Lecture 3b:

Trajectory Tracking

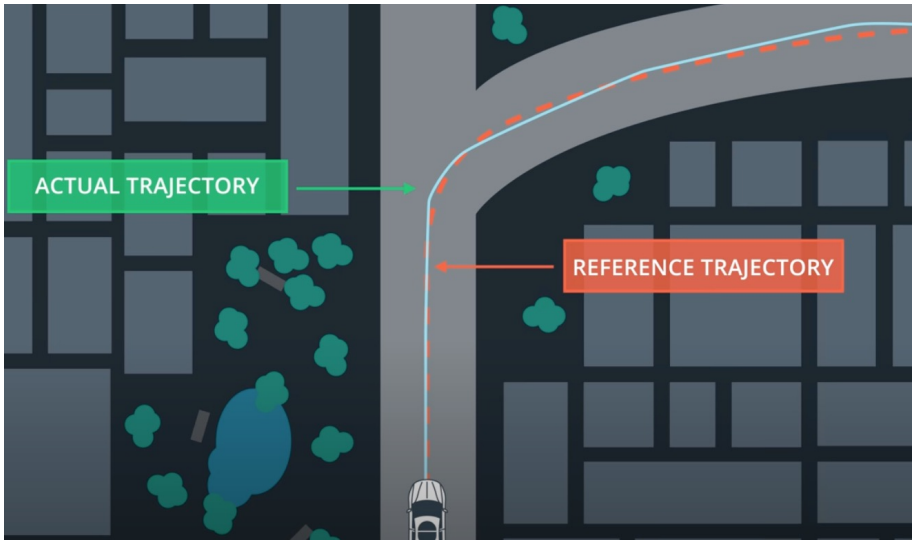
Slides heavily adapted from Alessandro de Luca (Sapienza)

Trajectory tracking



Trajectory tracking

In most cases, it is not only important to reach a goal, but also how we reach it.



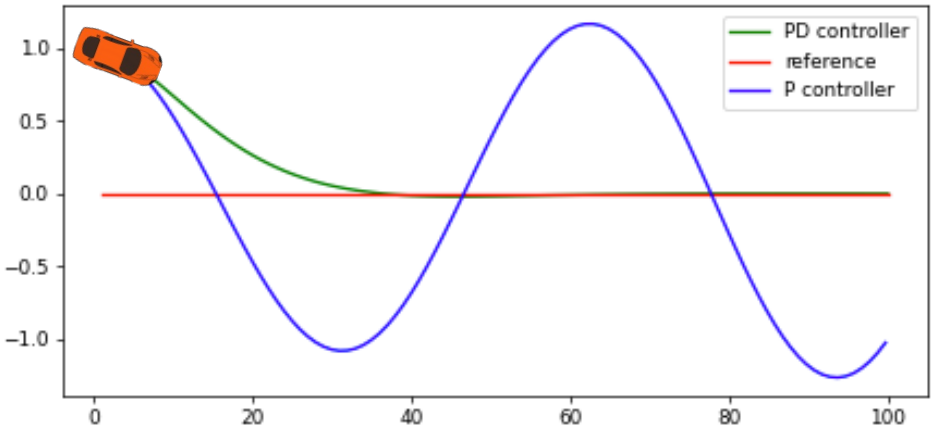
The ***desired trajectory*** $\boldsymbol{q}_d(t)$ (or in task space, $\boldsymbol{x}_d(t)$) is typically (continuously) **differentiable** up to a certain degree and it is designed to be feasible, i.e., compliant with the requirements for the system (e.g., actuation limits, desired comfort, …).

PD (PID) control

Similarly for the regulation problem, we could use a PD or PID controller to try and stabilize the system around the trajectory

- ***Feedback term***: $\boldsymbol{u}_{fb} = K_P(\boldsymbol{q}_d - \boldsymbol{q}) + K_D(\dot{\boldsymbol{q}}_d - \boldsymbol{q})$

- This **does not require a knowledge of the model**
- It is purely reactive (error-driven), so it **does not anticipate the erro**r
- We can add an **integral term to compensate a steady-state error**



Dynamic inversion

- Let's consider a robot with the canonical **dynamic model**

$$B(\boldsymbol{q})\ddot{\boldsymbol{q}} + \underbrace{C(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q})}_{n(\boldsymbol{q},\dot{\boldsymbol{q}})} = \boldsymbol{u}$$

- And a twice differentiable **desired trajectory** for $t \in [0, T]$

$$\boldsymbol{q}_d(t) \rightarrow \dot{\boldsymbol{q}}_d(t), \ddot{\boldsymbol{q}}_d(t)$$

- By **inverting the dynamics**, we can compute the input that in nominal conditions, and starting from the desired state, tracks the trajectory

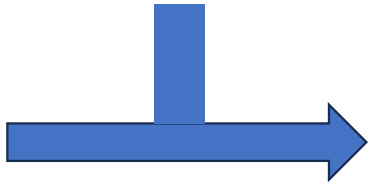
Dynamic inversion

trajectory

$$\boldsymbol{q}_d(t), \dot{\boldsymbol{q}}_d(t), \ddot{\boldsymbol{q}}_d(t)$$

model

$$B(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{n}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \boldsymbol{u}$$



Dynamic inversion

Feedforward input

$$\boldsymbol{u}_{ff} = B(\boldsymbol{q}_d)\ddot{\boldsymbol{q}}_d + \boldsymbol{n}(\boldsymbol{q}_d, \dot{\boldsymbol{q}}_d)$$

- The **feedforward input** allows the robot to follow the desired trajectory **if** it starts exactly on the **initial state of the trajectory**, i.e., $\boldsymbol{q}(0) = \boldsymbol{q}_d(0), \dot{\boldsymbol{q}}_d(0) = \ddot{\boldsymbol{q}}_d(0)$, the **model is known perfectly** (no unmodeled dynamics, perfect knolwedge of the parameters) and there are **no disturbances**
- In practice this is not realistic and we also need **feedback information** (e.g., PD or PID).

Feedforward + PD (PID)

We can combine the imperfect feedforward control with a PD control

- ***Feedforward term***: $\boldsymbol{u}_{ff} = \hat{B}(\boldsymbol{q}_d)\ddot{\boldsymbol{q}}_d + \hat{\boldsymbol{n}}(\boldsymbol{q}_d, \dot{\boldsymbol{q}}_d)$, with $\hat{B}$ and $\hat{\boldsymbol{n}}$ being estimates of the terms in the dynamic model
- ***Feedback term***: $\boldsymbol{u}_{fb} = K_P(\boldsymbol{q}_d - \boldsymbol{q}) + K_D(\dot{\boldsymbol{q}}_d - \boldsymbol{q})$, does not require a knowledge of the models

- The **feedforward + PD control** is

$$\boldsymbol{u} = \boldsymbol{u}_{ff} + \boldsymbol{u}_{fb}$$

  - ✓ Faster response than pure PD or PID
  - ✓ More robust than simple feedforward

Feedback linearization

Inverse dynamic compensation

The feedforward action based on dynamic inversion is used to **compensate** the nonlinear dynamics.

Feedback linearization

Use the control action to **cancel** the nonlinearities and transform the nonlinear system into an equivalent linear system.

Feedback linearization

Consider the nonlinear system

$$\dot{x} = f(x) + g(x)u$$
$$y = h(x)$$

The idea of ***feedback linearization*** is to find a control in the form

$$u = a(x) + b(x)v$$

So that the input-output response between v (***virtual input***) and y is **linear**.

Feedback linearization

$$\boldsymbol{n}(\boldsymbol{q},\dot{\boldsymbol{q}}) = (\boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}))$$

Robot model: $B(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{n}(\boldsymbol{q},\dot{\boldsymbol{q}}) = \boldsymbol{u}$

Linearizing input: $\boldsymbol{u} = \boldsymbol{n}(\boldsymbol{q},\dot{\boldsymbol{q}}) + B(\boldsymbol{q})\boldsymbol{v}$

$$\Rightarrow \quad \ddot{\boldsymbol{q}} = \boldsymbol{v}$$

- **The virtual input can then be chosen to stabilize the linear system**. Easier to do than trying to find a general nonlinear control.
- We can set the virtual control to:

$$\boldsymbol{v} = \ddot{\boldsymbol{q}}_d + K_P(\boldsymbol{q}_d - \boldsymbol{q}) + K_d(\dot{\boldsymbol{q}}_d - \dot{\boldsymbol{q}})$$

Feedback linearization

Finally, we can consider the **error dynamics** for the feedback linearized system

error $\boldsymbol{e} = \begin{bmatrix} \boldsymbol{q} - \boldsymbol{q}_d \\ \dot{\boldsymbol{q}} - \dot{\boldsymbol{q}}_d \end{bmatrix}$ ⟹ **Closed-loop error dynamics** $\dot{\boldsymbol{e}} = \begin{bmatrix} \dot{\boldsymbol{q}} - \dot{\boldsymbol{q}}_d \\ \ddot{\boldsymbol{q}} - \ddot{\boldsymbol{q}}_d \end{bmatrix} = \begin{bmatrix} \dot{\boldsymbol{q}} - \dot{\boldsymbol{q}}_d \\ \boldsymbol{v} - \ddot{\boldsymbol{q}}_d \end{bmatrix}$

$$= \begin{bmatrix} \dot{\boldsymbol{q}} - \dot{\boldsymbol{q}}_d \\ \cancel{\ddot{\boldsymbol{q}}_d} + K_P(\boldsymbol{q}_d - \boldsymbol{q}) + K_d(\dot{\boldsymbol{q}}_d - \dot{\boldsymbol{q}}) - \cancel{\ddot{\boldsymbol{q}}_d} \end{bmatrix}$$

$$= \begin{bmatrix} 0 & I \\ -K_p & -K_D \end{bmatrix} \boldsymbol{e} = A\boldsymbol{e}$$

- **We choose $\boldsymbol{K_P}, \mathbf{K_D}$ to place the poles of $\boldsymbol{A}$**. If $K_D = k_D I$ and $K_P = k_P I$, the eigenvalues of A are $\lambda = \frac{-k_D \pm \sqrt{k_D^2 - 4k_P}}{2}$, so the system is globally stable with $k_D > 0, 0 < k_P < \frac{k_D^2}{4}$

Dynamic inversion vs. Feedback linearization

Dynamic inversion (feedforward + feedback)

$$\boldsymbol{u} = \underbrace{\hat{B}(\boldsymbol{q}_d)\ddot{\boldsymbol{q}}_d + \hat{\boldsymbol{n}}(\boldsymbol{q}_d, \dot{\boldsymbol{q}}_d)}_{\text{Feedforward input}} + \underbrace{K_P(\boldsymbol{q}_d - \boldsymbol{q}) + K_D(\dot{\boldsymbol{q}}_d - \boldsymbol{q})}_{\text{Feedback}}$$

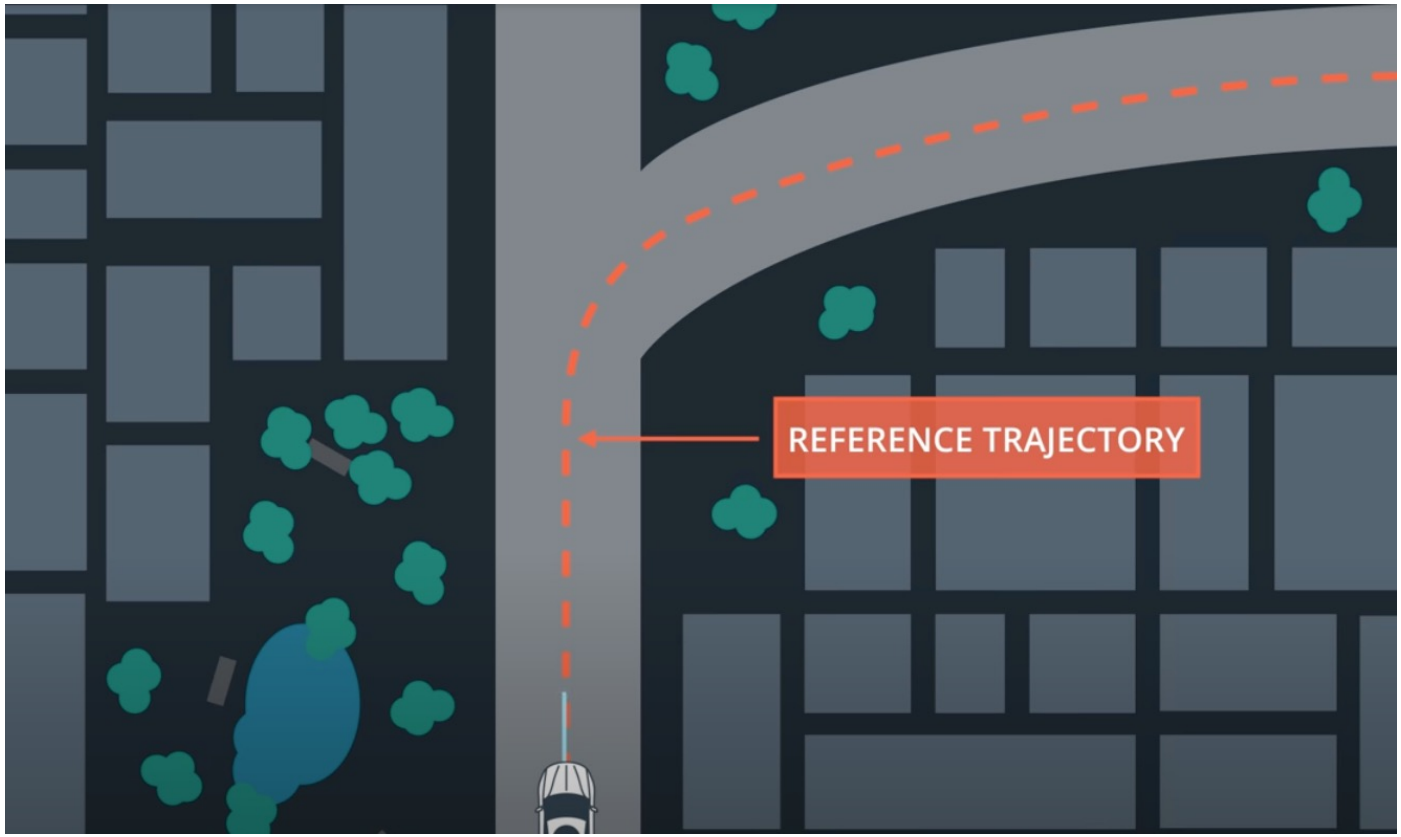
Feedback linearization

$$\boldsymbol{u} = \hat{B}(\boldsymbol{q}_d)[\underbrace{\ddot{\boldsymbol{q}}_d + K_P(\boldsymbol{q}_d - \boldsymbol{q}) + K_D(\dot{\boldsymbol{q}}_d - \boldsymbol{q})}_{\text{Virtual input}}] + \hat{\boldsymbol{n}}(\boldsymbol{q}_d, \dot{\boldsymbol{q}}_d)$$

- The feedback linearization is more robust to small uncertainties/disturbances

Example: car trajectory tracking

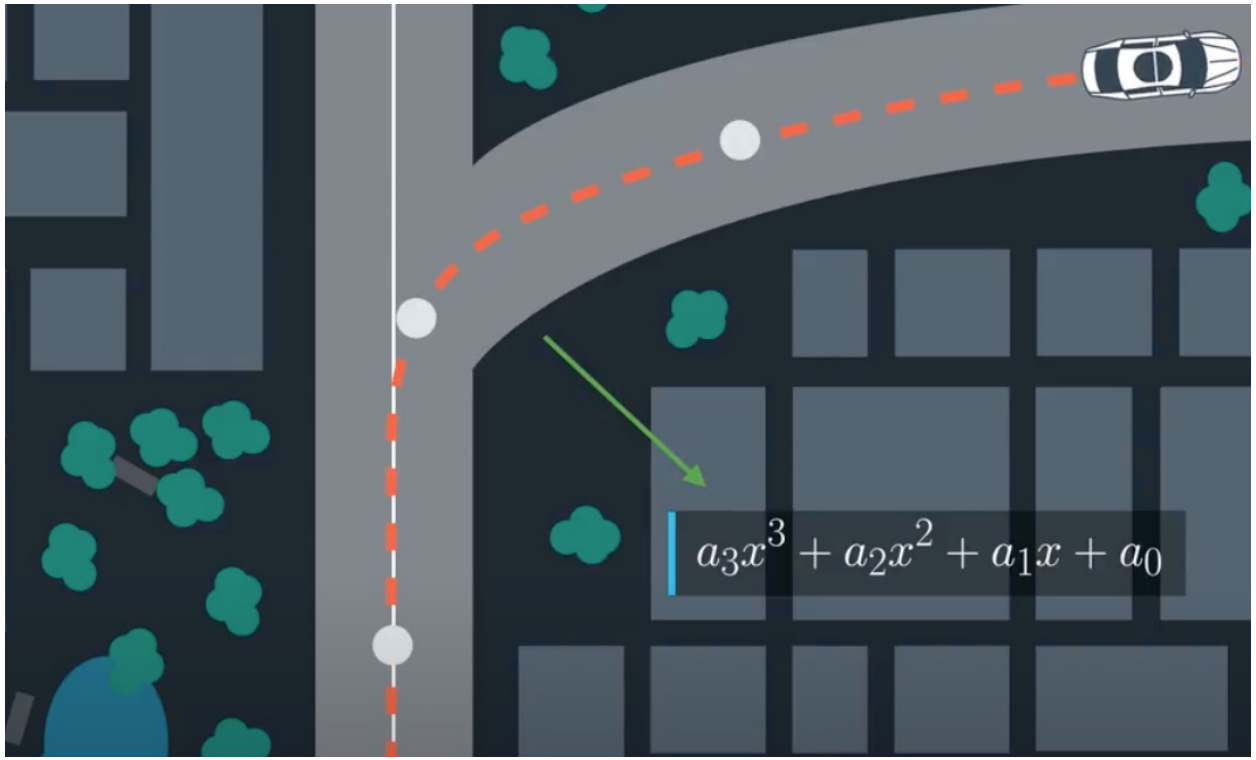
With a car-like robot, we may want to follow a planned course, with a **desired speed** (e.g. a cruise speed) and keeping the close to a **path** (e.g., center lane).



Example: car-like

With a car-like robot, we may want to follow a planned course, with a **desired speed** (e.g. a cruise speed) and keeping the close to a **path** (e.g., center lane).

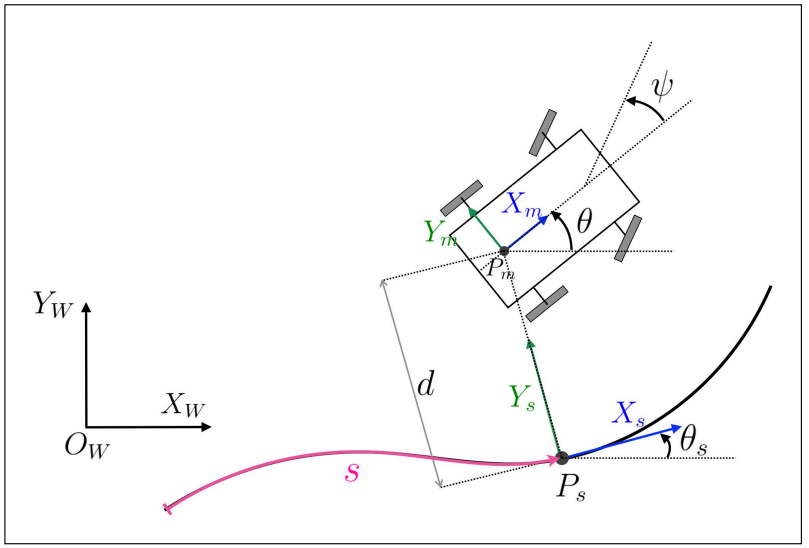
The trajectory is often decomposed in a **geometric path**, $\boldsymbol{r}(z) = [x(z), y(z)]$ (usually polynomial functions), and a **timing-law** $z(t)$



Example: car trajectory tracking

Kinematics in Frénet Frame

$$\begin{bmatrix} \dot{s} \\ \dot{d} \\ \dot{\theta}_e \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} \frac{\cos\theta_e}{1-d\kappa(s)} & 0 \\ \sin\theta_e & 0 \\ -\frac{\kappa(s)\cos\theta_e}{1-d\kappa(s)} + \frac{\tan\psi}{l} & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \delta \end{bmatrix}$$



- v **is the velocity**. In practice, in a car our input is not directly the velocity, but an acceleration (extreme simplification). So we can consider the additional dynamic

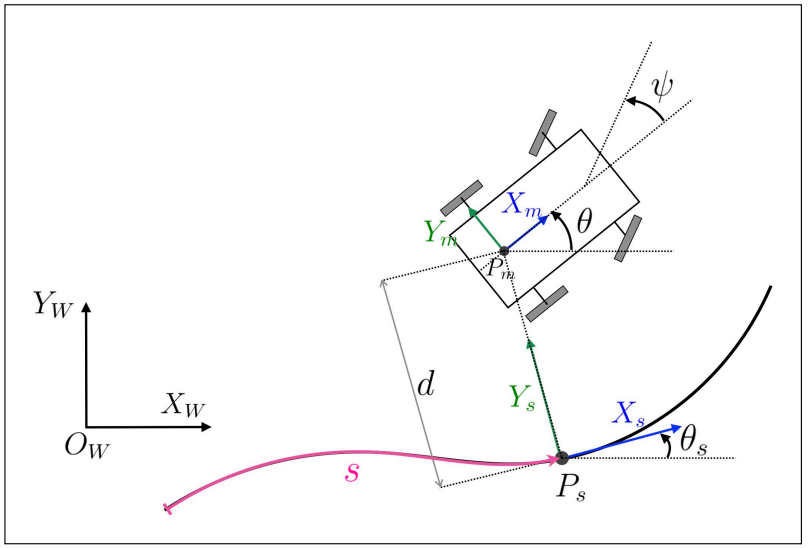
$$\dot{v} = a$$

  where a is the ***acceleration input***
- δ is the ***steering input***.

Example: car trajectory tracking

Kinematics in Frénet Frame

$$\begin{bmatrix} \dot{s} \\ \dot{d} \\ \dot{\theta}_e \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} \frac{\cos\theta_e}{1-d\kappa(s)} & 0 \\ \sin\theta_e & 0 \\ -\frac{\kappa(s)\cos\theta_e}{1-d\kappa(s)} + \frac{\tan\psi}{l} & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \delta \end{bmatrix}$$



We can use the steering input to steer the robot to follow the path and regulate to zero the ***lateral error*** d with a **PID controller**

$$\delta = -K_P d - K_D \dot{d} - K_I \int d\, d\tau$$

Lateral controller

What happens when we regulate the lateral error to zero, i.e., $d = \dot{d} = 0$?

Example: car trajectory tracking

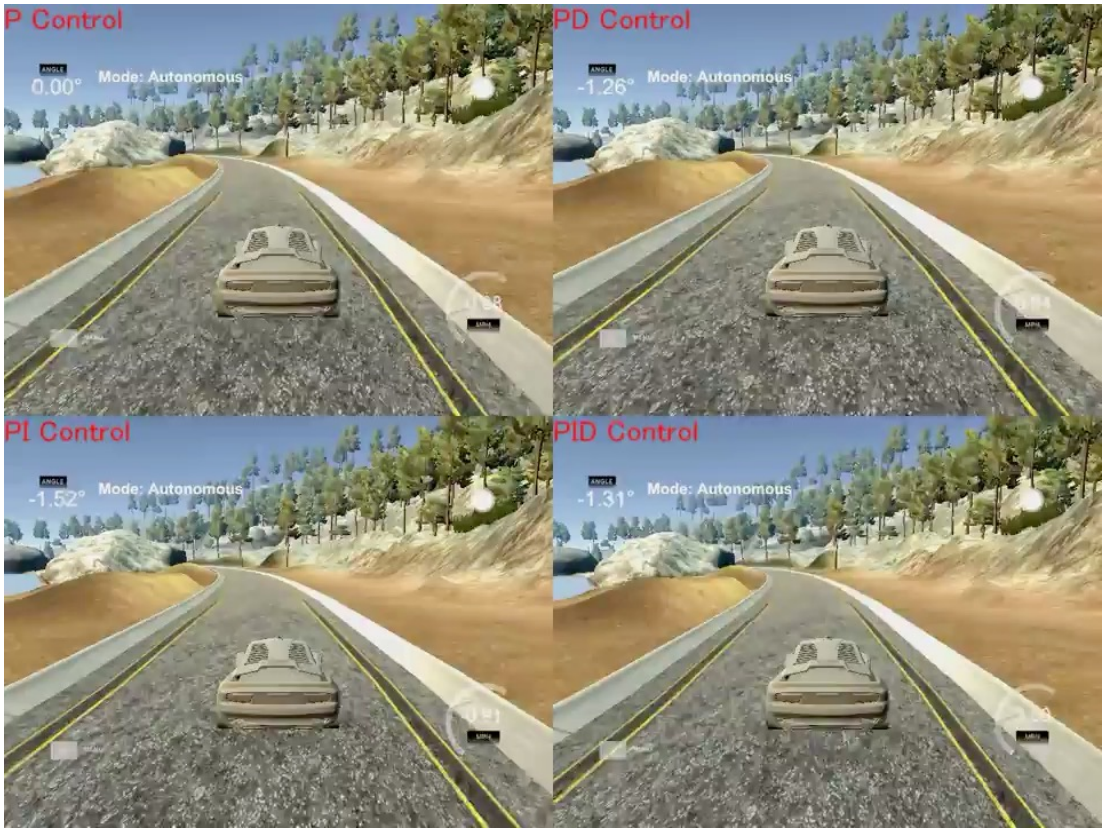
- We can use the acceleration input (gas pedal) to make the robot follow the path with the **desired velocity** $v_d(t)$

$$a = K_P(v_d - v) + K_D(\dot{v}_d - \dot{v}) + K_I \int (v_d - v) d\tau$$

Longitudinal control

- When the car is tracking the path, then $\dot{s} = v$
- If we want to have a fixed cruise speed, then $v_d = \text{const}$ and $\dot{v}_d = 0$

Example: car trajectory tracking



Example: UAV trajectory tracking

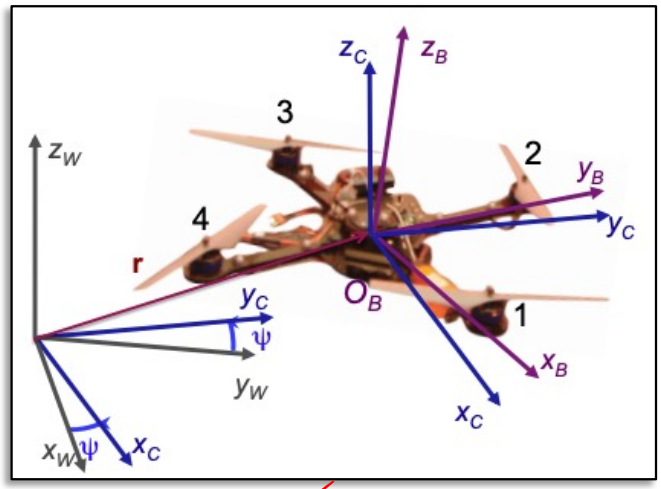
Dynamic model

$$m\ddot{\boldsymbol{p}} = -mg\boldsymbol{e}_3 + R(\phi,\theta,\psi)T$$
$$I\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega}\times I\boldsymbol{\omega} + \boldsymbol{\tau}$$

$$R = \begin{bmatrix} \mathrm{c}\,\theta\,\mathrm{c}\,\psi & \mathrm{c}\,\psi\,\mathrm{s}\,\theta\,\mathrm{s}\,\phi - \mathrm{c}\,\phi\,\mathrm{s}\,\psi & \mathrm{c}\,\phi\,\mathrm{s}\,\theta\,\mathrm{c}\,\psi + \mathrm{s}\,\phi\,\mathrm{s}\,\psi \\ \mathrm{c}\,\theta\,\mathrm{s}\,\psi & \mathrm{s}\,\psi\,\mathrm{s}\,\theta\,\mathrm{s}\,\phi + \mathrm{c}\,\phi\,\mathrm{c}\,\psi & \mathrm{c}\,\phi\,\mathrm{s}\,\theta\,\mathrm{s}\,\psi - \mathrm{s}\,\phi\,\mathrm{c}\,\psi \\ -\mathrm{s}\,\theta & \mathrm{c}\,\theta\,\mathrm{s}\,\phi & \mathrm{c}\,\theta\,\mathrm{c}\,\phi \end{bmatrix}$$

$$\begin{bmatrix} \dot{\psi} \\ \dot{\theta} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} 0 & \sin\phi\sec\theta & \cos\phi\sec\theta \\ 0 & \cos\phi & -\sin\phi \\ 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \end{bmatrix} \begin{bmatrix} p \\ q \\ p \end{bmatrix}$$

$\dot{\boldsymbol{\eta}}$ $\qquad M(\psi,\theta,\phi) \qquad$ $\boldsymbol{\omega}$



Input $\boldsymbol{u} = \begin{bmatrix} T \\ \tau_1 \\ \tau_2 \\ \tau_3 \end{bmatrix}$ → ***Thrust*** / ***Torques***

Output to track $\boldsymbol{r}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \\ \psi(t) \end{bmatrix}$ → ***Position*** / ***Yaw***

Example: UAV trajectory tracking

Let's consider as state the vector $\boldsymbol{x} = \begin{bmatrix} x & y & z & \phi & \theta & \psi & \dot{x} & \dot{y} & \dot{z} & p & q & r \end{bmatrix}^T \in \mathbb{R}^{12\times 1}$

position orientation velocity Angular velocity

$\dot{x} = f(u) + f(x)u$

Then, we can write the state space model $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{g}(\boldsymbol{x})\boldsymbol{u}$

$$\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ f_{(4,1)} \\ f_{(5,1)} \\ f_{(6,1)} \\ 0 \\ 0 \\ g \\ \frac{Iy-Iz}{Ix}qr \\ \frac{Iz-Ix}{Iy}pr \\ \frac{Ix-Iy}{Iz}pq \end{bmatrix}, \boldsymbol{g}(\boldsymbol{x}) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ g_{(7,1)} & 0 & 0 & 0 \\ g_{(8,1)} & 0 & 0 & 0 \\ g_{(9,1)} & 0 & 0 & 0 \\ 0 & \frac{1}{Ix} & 0 & 0 \\ 0 & 0 & \frac{1}{Iy} & 0 \\ 0 & 0 & 0 & \frac{1}{Iz} \end{bmatrix}$$

$$\begin{cases} f_{(4,1)} = p + q\sin\phi\tan\theta + r\cos\phi\tan\theta \\ f_{(5,1)} = q\cos\phi - r\cos\phi \\ f_{(6,1)} = q\sin\phi\sec\theta + r\cos\phi\sec\theta \\ g_{(7,1)} = -\frac{1}{m}(\cos\phi\cos\psi\sin\theta + \sin\phi\sin\psi) \\ g_{(8,1)} = -\frac{1}{m}(\cos\phi\sin\psi\sin\theta - \sin\phi\cos\psi) \\ g_{(9,1)} = -\frac{1}{m}(\cos\phi\cos\theta)\,. \end{cases}$$

This can be **feedback linearized**, with a trick

Example: UAV trajectory tracking

To feedback linearize the system, we use the trick called **dynamic extension**:

- Instead of $\boldsymbol{u}$, we take a **new input**

$$\bar{\boldsymbol{u}} = [\ddot{T} \quad \tau_1 \quad \tau_2 \quad \tau_3\ ]^T$$

- The thrust T and its derivative $\dot{T}$ dynamically extend the state

---

It can be shown (not in this lecture), that with this trick we can write the system as

$$\begin{bmatrix} \dddot{x} \\ \dddot{y} \\ \dddot{z} \\ \ddot{\psi} \end{bmatrix} = \begin{bmatrix} n_x \\ n_y \\ n_z \\ n_\psi \end{bmatrix} + \begin{bmatrix} B_x \\ B_y \\ B_z \\ B_\psi \end{bmatrix} \bar{\boldsymbol{u}}$$

Linear input-output (**regular form**) where

$$\begin{bmatrix} n_x \\ n_y \\ n_z \end{bmatrix} = -\frac{TS(R\boldsymbol{e}_3)RI^{-1}S(\boldsymbol{\omega})I\boldsymbol{\omega} + RS(\dot{\boldsymbol{\eta}})^2\boldsymbol{e}_3T + 2RS(\dot{\boldsymbol{\eta}})\boldsymbol{e}_3\dot{T}}{m}$$

$$n_\psi = \left[\dot{M}\omega - MI^{-1}S(\omega)I\omega\right]_3$$

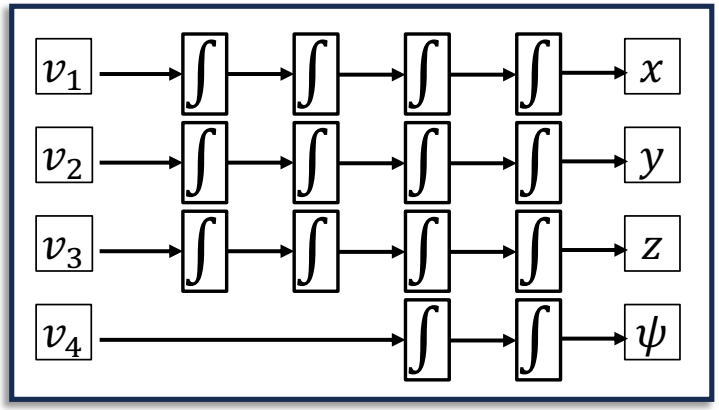
$$\begin{bmatrix} B_x \\ B_y \\ B_z \end{bmatrix} = \left[-\frac{R\boldsymbol{e}_3}{m} \quad -\frac{TS(R\boldsymbol{e}_3)RI^{-1}}{m}\right]$$

$$B_\psi = [0_{3\times1} \quad MI^{-1}]_3$$

Example: UAV trajectory tracking

With the system in regular form, we can finally do the feedback linearization

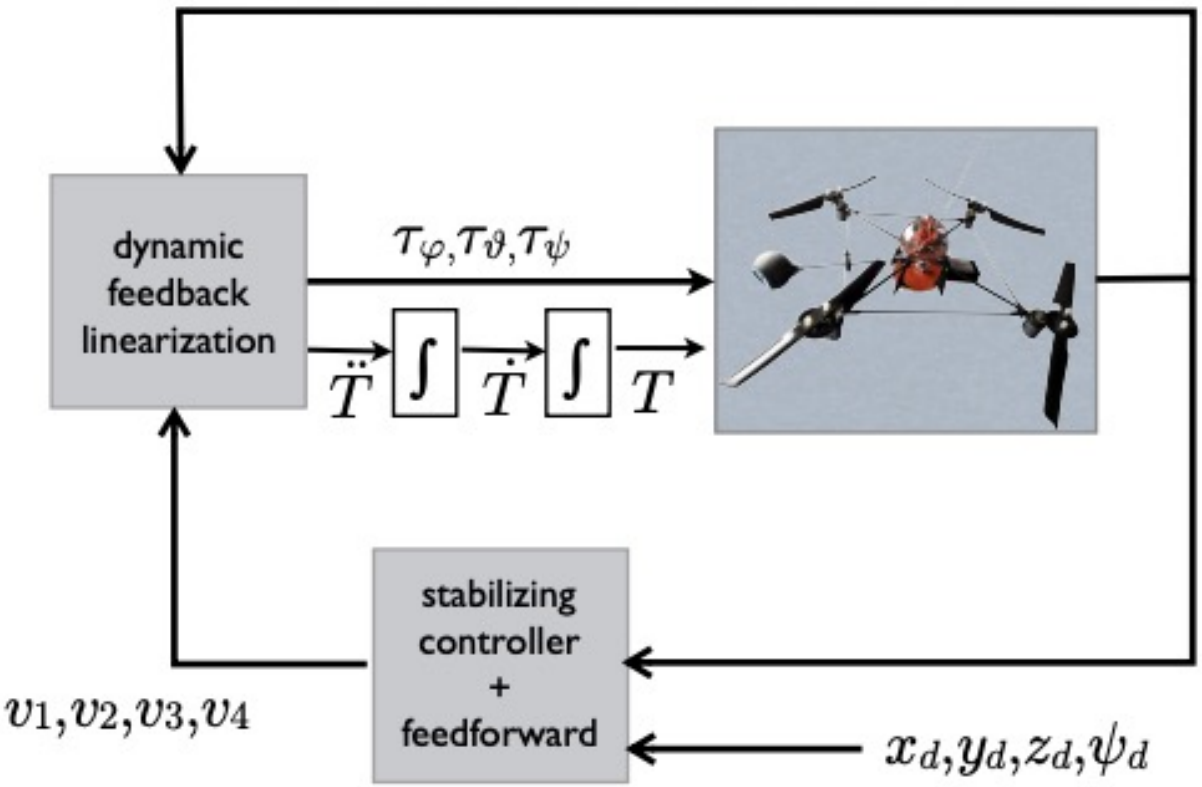
$$\begin{bmatrix} \dddot{x} \\ \dddot{y} \\ \dddot{y} \\ \ddot{\psi} \end{bmatrix} = \underbrace{\begin{bmatrix} n_x \\ n_y \\ n_z \\ n_\psi \end{bmatrix}}_{\boldsymbol{n}} + \underbrace{\begin{bmatrix} B_x \\ B_y \\ B_z \\ B_\psi \end{bmatrix}}_{B} \bar{\boldsymbol{u}} \quad \Rightarrow \quad \bar{\boldsymbol{u}} = B^{-1}[-\boldsymbol{n} + \boldsymbol{v}] \quad \Rightarrow$$



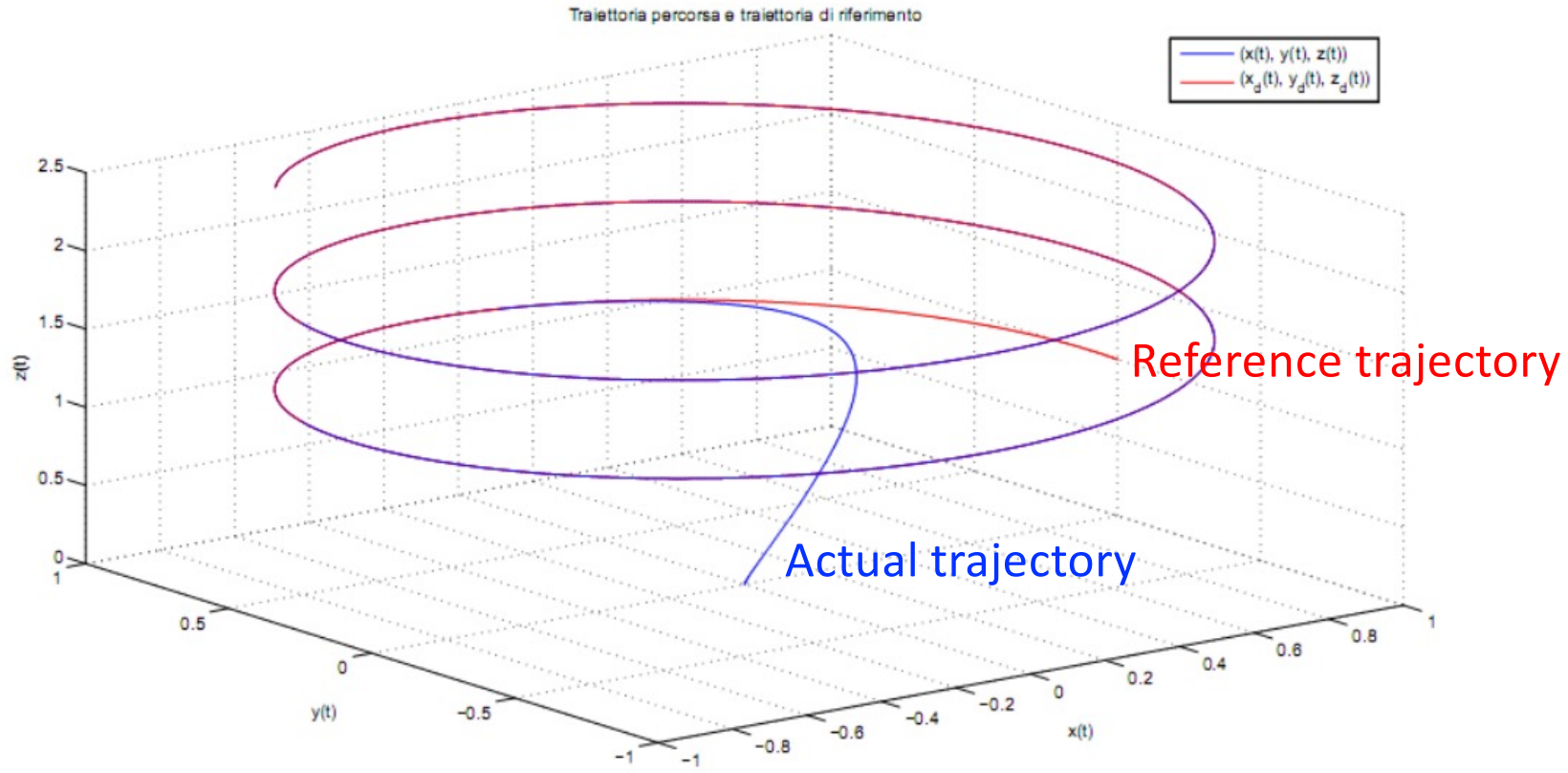
The system becomes like a chain of integrators, and we can choose the **virtual input** as a **PD-like control**

$$v_1 = \dddot{x}_d - k_3(\ddot{x} - \ddot{x}_d) - k_2(\dot{x} - \dot{x}_d) - k_1(\dot{x} - \dot{x}_d) - k_0(x - x_d)$$
$$v_2 = \dddot{y}_d - k_3(\ddot{y} - \ddot{y}_d) - k_2(\dot{y} - \dot{y}_d) - k_1(\dot{y} - \dot{y}_d) - k_0(y - y_d)$$
$$v_3 = \dddot{z}_d - k_3(\ddot{z} - \ddot{z}_d) - k_2(\dot{z} - \dot{z}_d) - k_1(\dot{z} - \dot{z}_d) - k_0(z - z_d)$$
$$v_4 = \ddot{\psi}_d - k_5(\dot{\psi} - \dot{\psi}_d) - k_0(\psi - \psi_d)$$

Example: UAV trajectory tracking



Example: UAV trajectory tracking



Example: UAV trajectory tracking

Here the UAV is tracking a path that is modified in real-time by a person

Example: UAV trajectory tracking

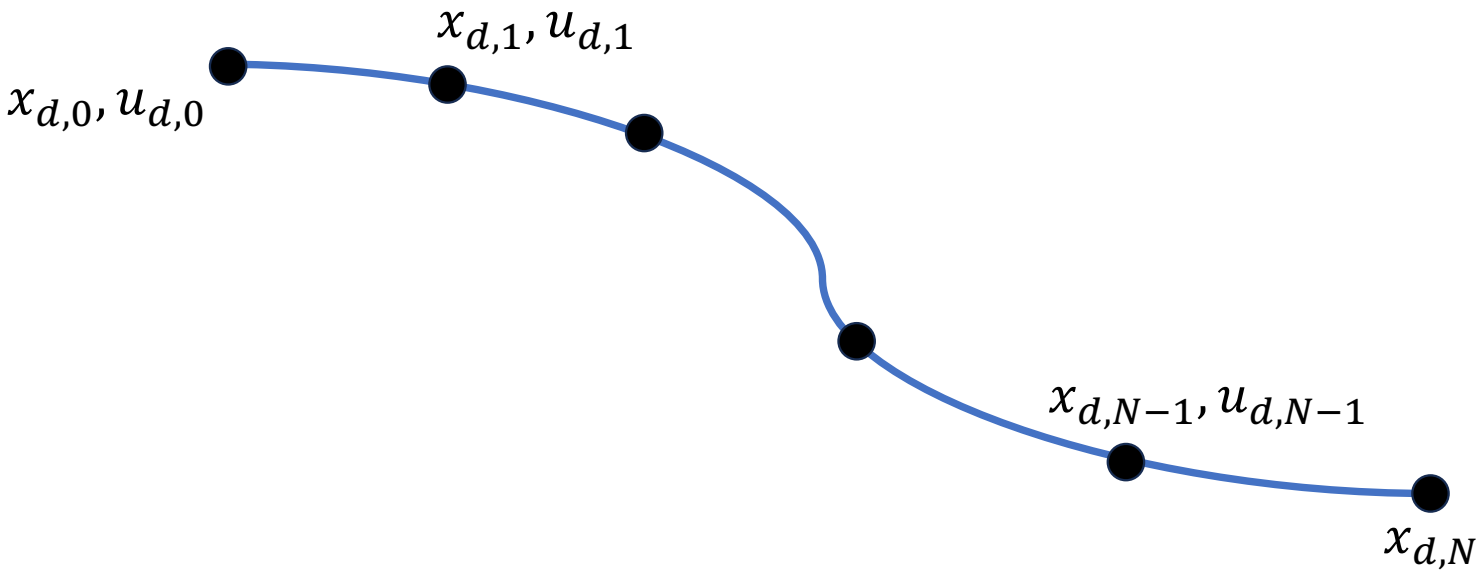
With the regular model of the UAV, we may not only do feedback linearization, but also use robust controllers like an Adaptive Super-Twisting controller (fancy name, we are not going to study it) [Source, S. Rajappa et al, "Adaptive Super Twisting Controller for a quadrotor UAV", ICRA 2016].

Optimal tracking

- These feedback control schemes are **purely reactive**. But we may also **use the model to try and optimize the tracking over a future time horizon**
  - LQR extension to trajectory tracking
  - Model Predictive Control (MPC)

LQR extension to trajectory tracking

- To extend the **LQR for trajectory tracking**, we use the same strategy seen for the regulation problem, but this time we must linearize the system around a trajectory



LQR extension to trajectory tracking

- Assume having a **feasible reference trajectory** (sampled in discrete time)

$$x_{d,0}, x_{d,1}, \dots, x_{d,N} \iff \exists u_{d,0}, u_{d,1}, \dots, u_{d,N-1} : x_{d,t+1} = f(x_{d,t}, u_{d,t})$$

- Linearize the system around the trajectory

$$x_{t+1} \approx f(x_{d,t}, u_{d,t}) + \underbrace{\left.\frac{\partial f}{\partial x}\right|_{x_{d,t}, u_{d,t}}}_{A_t} (x_t - x_{d,t}) + \underbrace{\left.\frac{\partial f}{\partial u}\right|_{x_{d,t}, u_{d,t}}}_{B_t} (u_t - u_{d,t})$$

$$x_{t+1} - x_{d,t+1} \approx A_t(x_t - x_{d,t}) + B_t(x_t - x_{d,t})$$

- Matrices A_t and B_t are **time varying**.

LQR extension to trajectory tracking

- Let's define for sake of clairty $\hat{x}_t = (x_t - x_{d,t})$ and $\hat{u}_t = (u_t - u_{d,t})$

$$\min_{\boldsymbol{u_d}} \sum_{t=0}^{N-1} [(x_t - x_{d,t})^T Q (x_t - x_{d,t}) + (u_t - u_{d,t})^T R (u_t - u_{d,t})] + (x_N - x_{d,N})^T Q_f (x_N - x_{d,N})$$

$$\text{s.t. } x_{t+1} - x_{d,t+1} \approx A_t (x_t - x_{d,t}) + B_t (u_t - u_{d,t})$$

- The optimal cost-to-go at the end of the trajectory is

$$V_N = \hat{x}_N^T P_N \hat{x}_N \text{ with } P_N = Q_f$$

LQR extension to trajectory tracking

- Iterating for $t = N - 1$

$$V_{N-1} = \min_{\hat{u}_{N-1}} \hat{x}_{N-1}^T Q\, \hat{x}_{N-1} + \hat{u}_{N-1}^T R \hat{u}_{N-1} + V_N(\hat{x}_N)$$

$$= \min_{\hat{u}_{N-1}} \hat{x}_{N-1}^T Q\, \hat{x}_{N-1} + \hat{u}_{N-1}^T R \hat{u}_{N-1} + V_N(A\hat{x}_{N-1} + B\hat{u}_{N-1})$$

$$= \min_{u_{N-1}} \hat{x}_{N-1}^T Q\, \hat{x}_{N-1} + \hat{u}_{N-1}^T R \hat{u}_{N-1} + (A\hat{x}_{N-1} + B\hat{u}_{N-1})^T Q_f (A\hat{x}_{N-1} + B\hat{u}_{N-1})$$

- If we solve this like in the regulation case (you can try at home), we find that

$$u_t^* = u_{d,t} \overbrace{- (R + B_t^T P_{t+1} B_t)^{-1} B_t^T P_{t+1} A_t}^{K_t} \left(x_t - x_{d,t}\right) = u_{d,t} + K_t \left(x_t - x_{d,t}\right)$$

$$P_t = Q + A_t^T (P_{t+1} - P_{t+1} B_t (R + B_t^T P_{t+1} B_t)^{-1} B_t^T P_{t+1}) A_t$$

Closed form solution, but the matrix of weights K changes at each step.

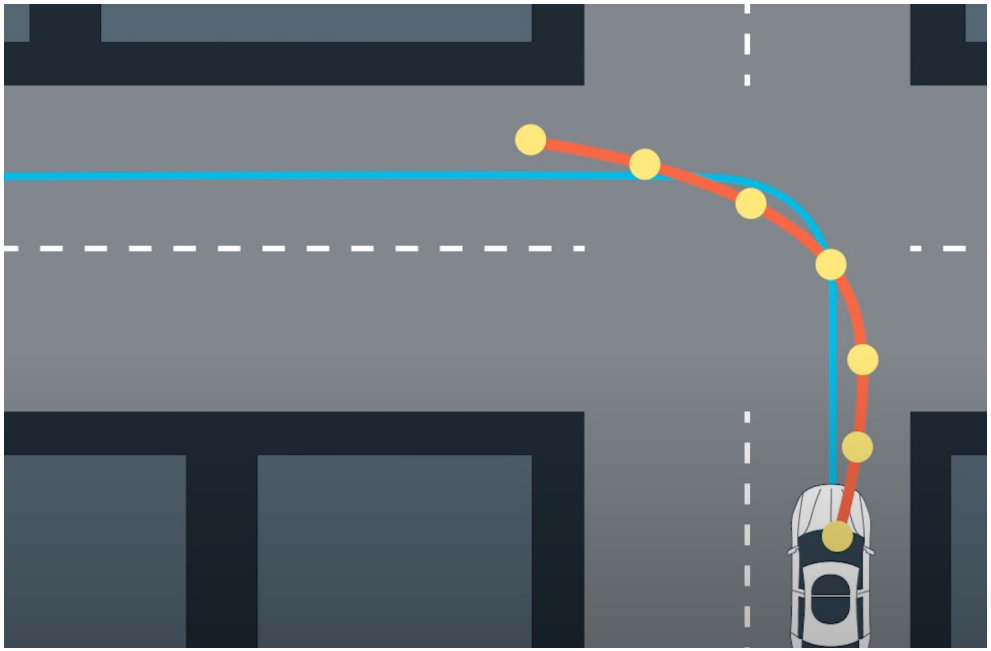
Model Predictive Control (MPC)

- **Pre-computing the optimal inputs** for the whole trajectory at the start, and just applying them **doesn't work well**
  - The world changes
  - We may have errors in our execution, because our internal model is not precise or we have noise

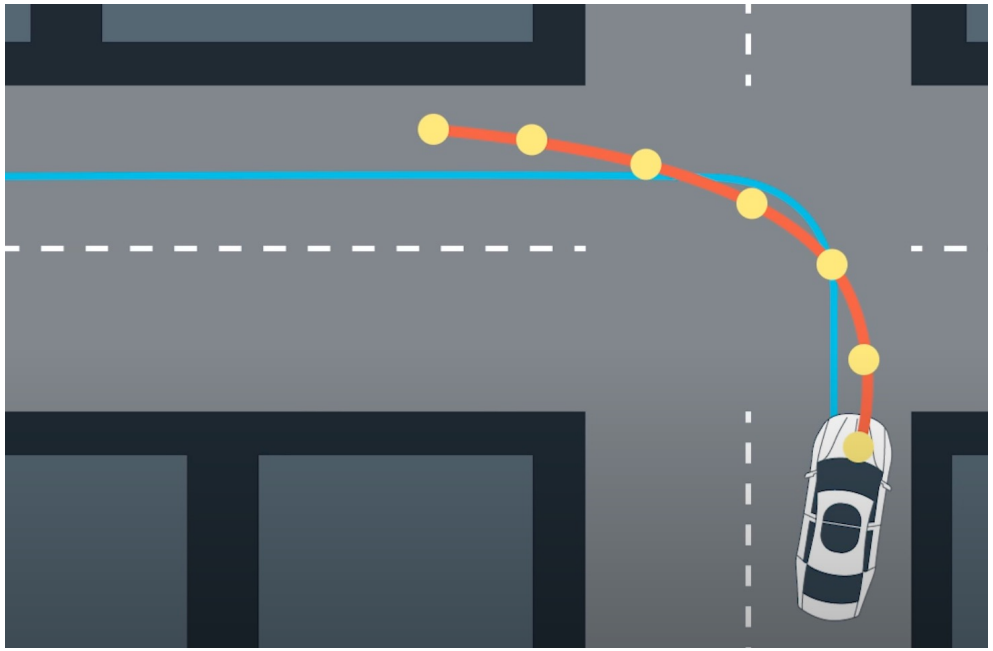
- We can solve the problem for a limited horizon, and **only apply the first control**. At the next step, we move the horizon, solve the problem again and apply the first input

Model Predictive Control (MPC)

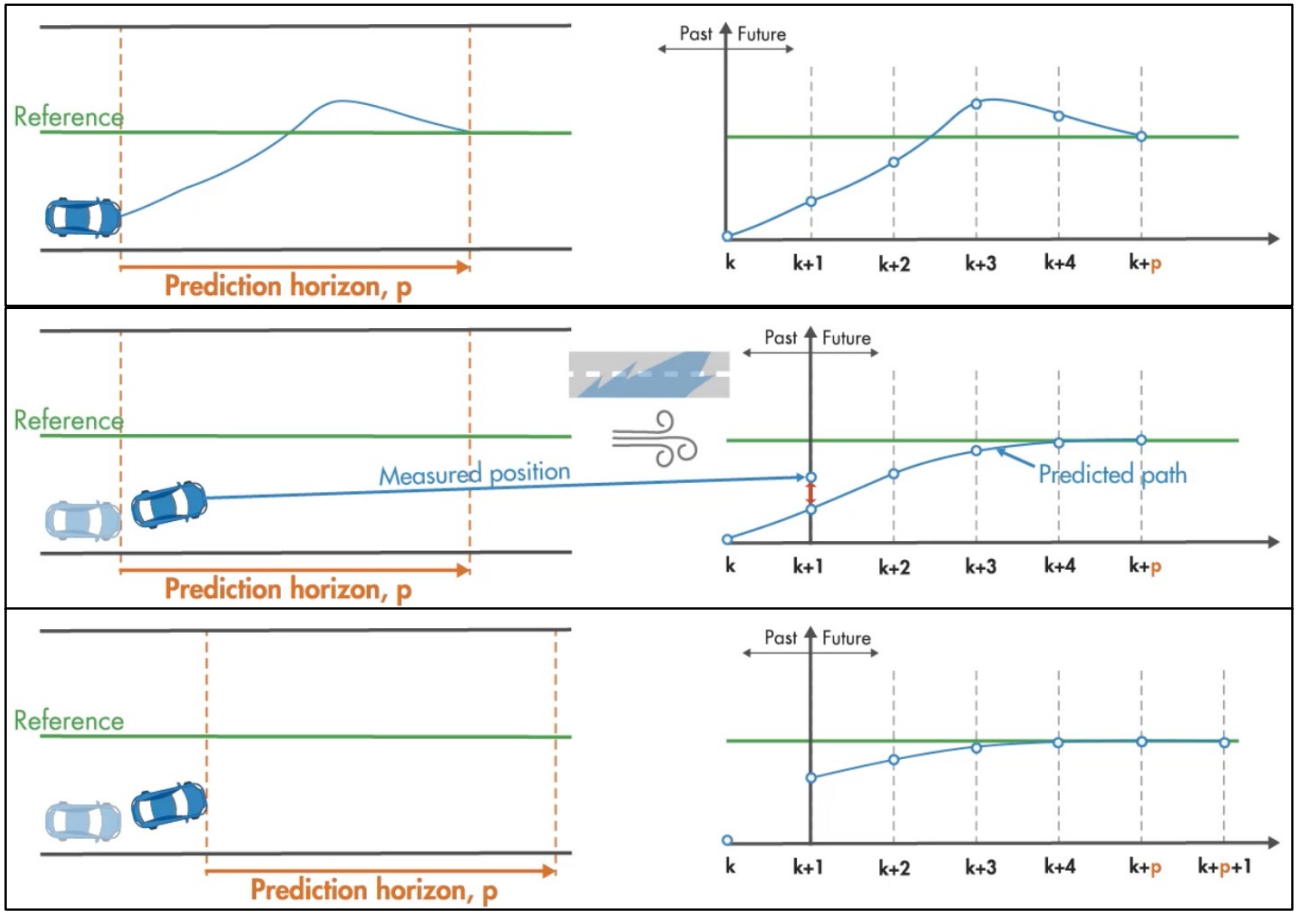
t_0: the robot computes the optimal control sequence on a finite horizon. Then it executes the first command.



t_1: the robot now re-computes the optimal control sequence on a finite horizon. **The new prediction may be different from the previous**

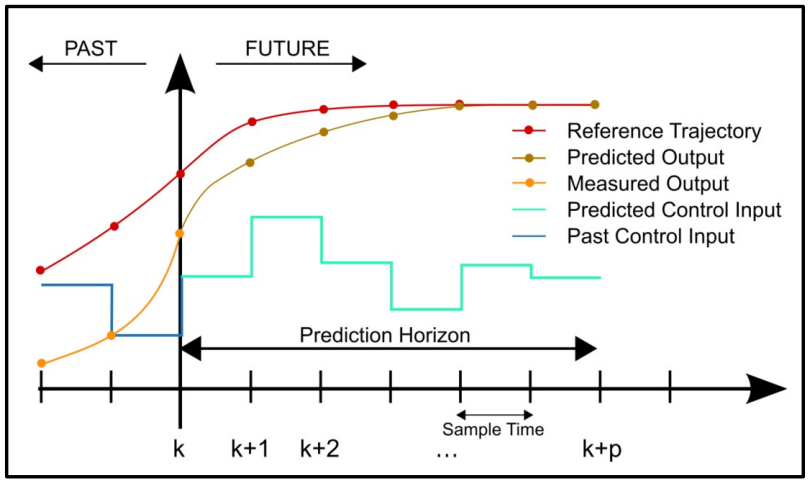


Model Predictive Control (MPC)



Model Predictive Control (MPC)

- Model Predictive Control (MPC) is a control approach which can handle ***state and input constraints***
- it solves at each time step an optimization problem over a fixed **prediction horizon**, computing an optimal sequence of control inputs
- only the **first input** is applied to the system, while the rest is discarded (**receding horizon** strategy)



Linear MPC

- We can use a linear MPC for trajectory tracking, by linearizing the model of the robot around the reference trajectory

MPC Optimization problem

$$\min_{[\hat{u}_0,\dots,\hat{u}_{N-1}]} J(\hat{\boldsymbol{x}},\hat{\boldsymbol{u}}) = \hat{x}_N^T Q_f \hat{x}_N + \sum_{k=0}^{N-1} \hat{x}_k^T Q \hat{x}_k + \hat{u}_k^T R \hat{u}_k$$

$$\text{s.t.} \quad \begin{aligned} \hat{\boldsymbol{x}}_{k+1} &= A_k \hat{\boldsymbol{x}}_k + B_k \hat{\boldsymbol{u}}_k & k &= 0,\dots,N-1 \\ \boldsymbol{x}_{\min} &< \boldsymbol{x}_k < \boldsymbol{x}_{\max} & k &= 1,\dots,N \\ \boldsymbol{u}_{\min} &< \boldsymbol{u}_k < \boldsymbol{u}_{\max}. & k &= 0,\dots,N-1 \end{aligned}$$

- This is still a ***quadratic problem***, but in general it does not have a closed form solution and it is solved numerically

LQR vs. Linear MPC

LQR	***MPC***
• Finite or infinite horizon	• Limited horizon (typically small)
• Explicit linear solution	• In general, no explicit solution. **Numerical optimization solvers** are used (e.g., ipopt).
• Does not handle constraint	• Handles constraint
• **Offline optimal control**. Input sequence computed at once	• **Online optimal control**. Only the first input is used. Therefore, it must be recomputed at each step

MPC makes no assumptions about linearity, so it can handle hard constraints and it is more robust w.r.t. deviations from the linearized operating point. The fully ***nonlinear MPC*** has even better performance.

Non-Linear MPC (NMPC)

- **MPC for a linear approximation** of a nonlinear system **works if we start close to the desired trajectory**
- We can directly solve the nonlinear model, and even include nonlinear constraints

$$\min_{[\hat{u}_0,\dots,\hat{u}_{N-1}]} J(\hat{\boldsymbol{x}},\hat{\boldsymbol{u}}) = s(\hat{\boldsymbol{x}}_N) + \sum_{k=0}^{N-1} l(\hat{\boldsymbol{x}}_k,\hat{\boldsymbol{u}}_k)$$

The cost function is usually quadratic

NMPC optimization problem

$$\begin{aligned}
&\hat{\boldsymbol{x}}_{k+1} = f(x_k) + g(x_k)u && k = 0,\dots,N-1\\
&\boldsymbol{x}_{\min} < \boldsymbol{x}_k < \boldsymbol{x}_{\max} && k = 1,\dots,N\\
\text{s.t.}\quad &\boldsymbol{u}_{\min} < \boldsymbol{u}_k < \boldsymbol{u}_{\max}. && k = 0,\dots,N-1\\
&m(x_k) = 0. && k = 0,\dots,N-1\\
&l(u_k) = 0. && k = 0,\dots,N-1
\end{aligned}$$

Non-Linear MPC (NMPC)

NMPC Optimization problem

$$\min_{[\hat{u}_0,\dots,\hat{u}_{N-1}]} J(\hat{\boldsymbol{x}},\hat{\boldsymbol{u}}) = s(\hat{\boldsymbol{x}}_N) + \sum_{k=0}^{N-1} l(\hat{\boldsymbol{x}}_k,\hat{\boldsymbol{u}}_k)$$

s.t.

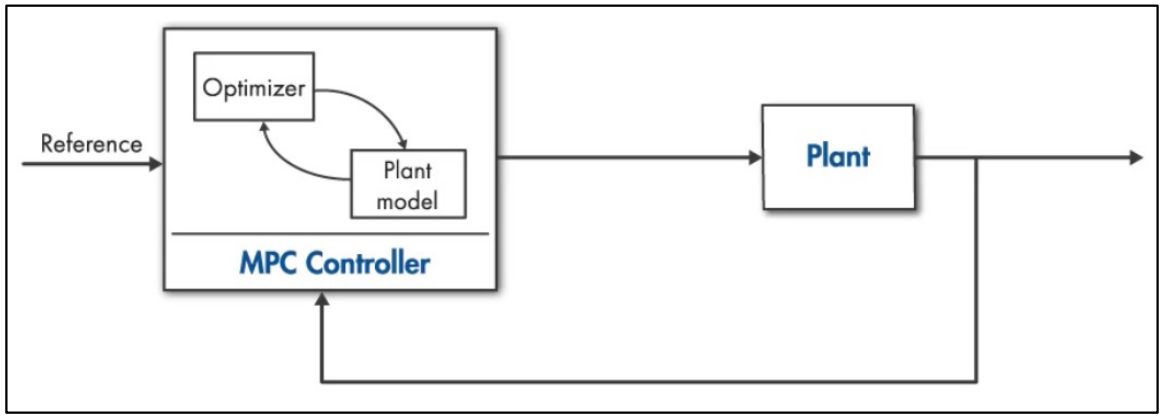
$$\hat{\boldsymbol{x}}_{k+1} = f(x_k) + g(x_k)u \qquad k = 0,\dots,N-1$$

…

…

- The general NMPC problem is not convex anymore
- The problem can have multiple **local optima**, and convergence to the **global optimum** is **not guaranteed**
- NMPC is more computationally intensive
- In practice it works very well

Optimizers



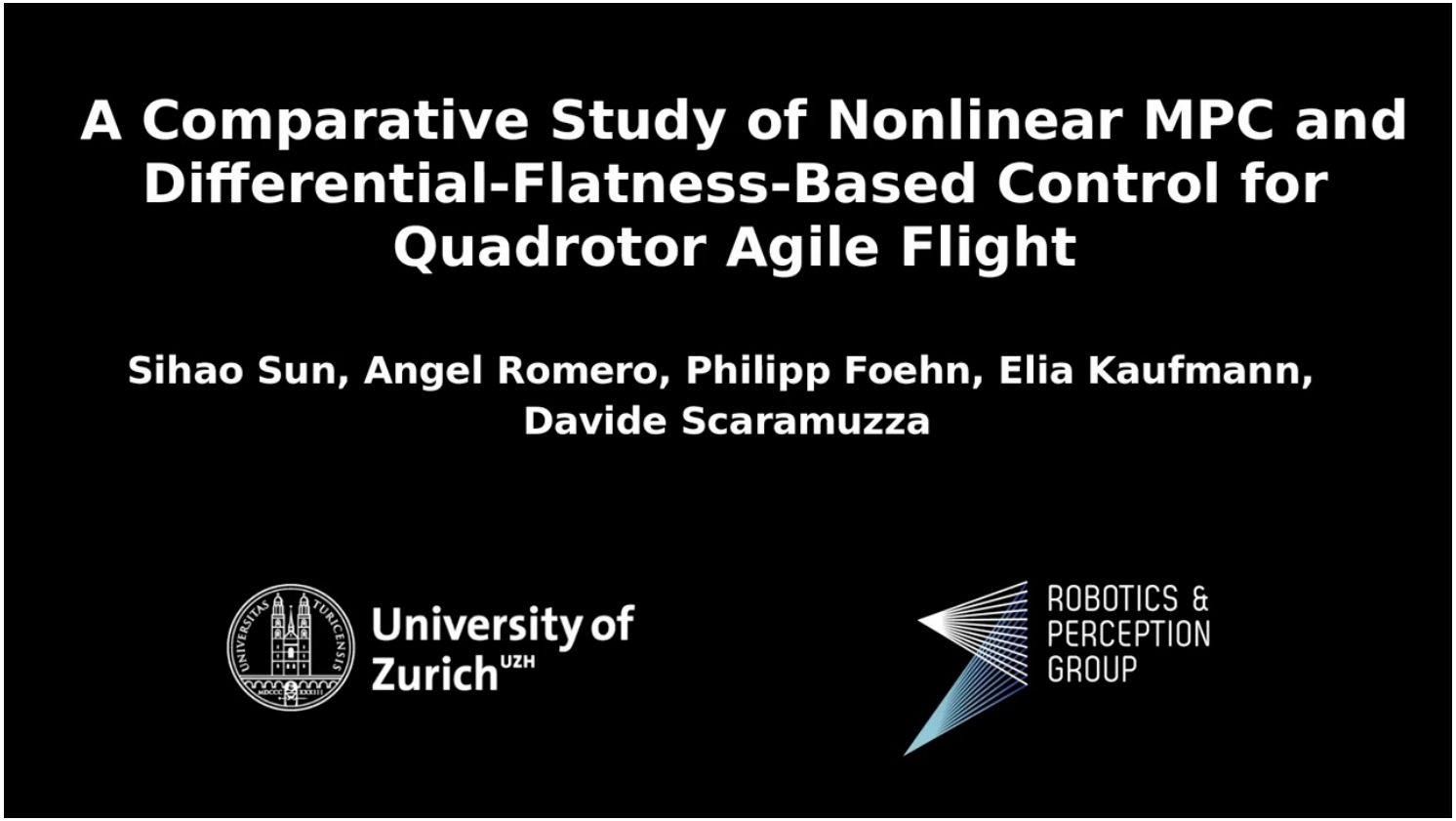
- There are various open-source suites of optimizers available, such as CasADi, which includes:
  - Algorithmic differentiation
  - Nonlinear and quadratic programming solvers
  - Python and C++ api

CasADi

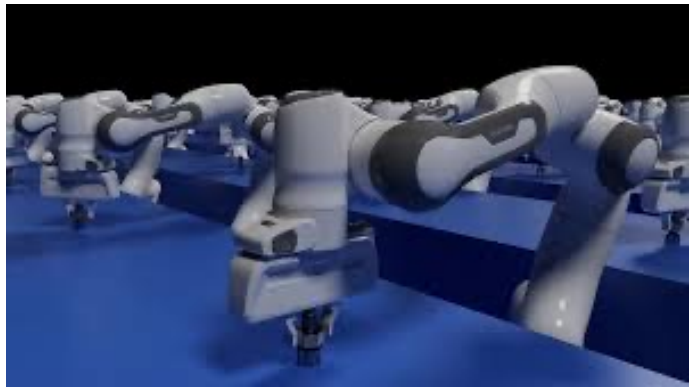
https://web.casadi.org/
https://youtu.be/JI-AyLv68Xs?si=4Hbrf7BV_N6WO8iw

Example: car tracking with MPC

Example: UAV tracking with NMPC



Contact motion



Contact motion

Working with manipulators, we often don't just want them to move, but we need them to move while in contact with the environment => **Contact forces**

Contact motion

The dynamic model of the robot, is extended to include the contact forces $\boldsymbol{f}$

$$B(\boldsymbol{q})\ddot{\boldsymbol{q}} + C(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{u} + J^T(q)\boldsymbol{f}$$

Since the forces $\boldsymbol{f}$ perform work on $[\boldsymbol{v},\boldsymbol{\omega}]$ but the trajectory is expressed as $[\boldsymbol{p},\boldsymbol{\phi}]$ and $[\dot{\boldsymbol{p}},\dot{\boldsymbol{\phi}}]$, it is convenient to use the **analytical Jacobian**

$$B_r(\boldsymbol{q})\ddot{\boldsymbol{r}} + C_r(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{r}} + \boldsymbol{g}_r(\boldsymbol{q}) = J_A^{-T}(q)\boldsymbol{u} + \boldsymbol{f}_r$$

where

$$\boldsymbol{f}_r = T(\boldsymbol{\phi})\boldsymbol{f}$$

$$\begin{bmatrix}\dot{\boldsymbol{p}}\\ \dot{\boldsymbol{\phi}}\end{bmatrix} = T(\boldsymbol{\phi})\begin{bmatrix}\boldsymbol{v}\\ \boldsymbol{\omega}\end{bmatrix}$$

and

$$B_r = J_A^{-T}BJ_A^{-1}$$
$$C_r = J_A^{-T}CJ_A^{-1}$$
$$\boldsymbol{g}_r = J_A^{-T}\boldsymbol{g}J_A^{-1}$$

Impedance control

Neville Hogan @MIT

The idea of ***impedance control*** is to impose that the system at contact behaves like a desired **mass-spring-damper** system

We can use feedback linearization

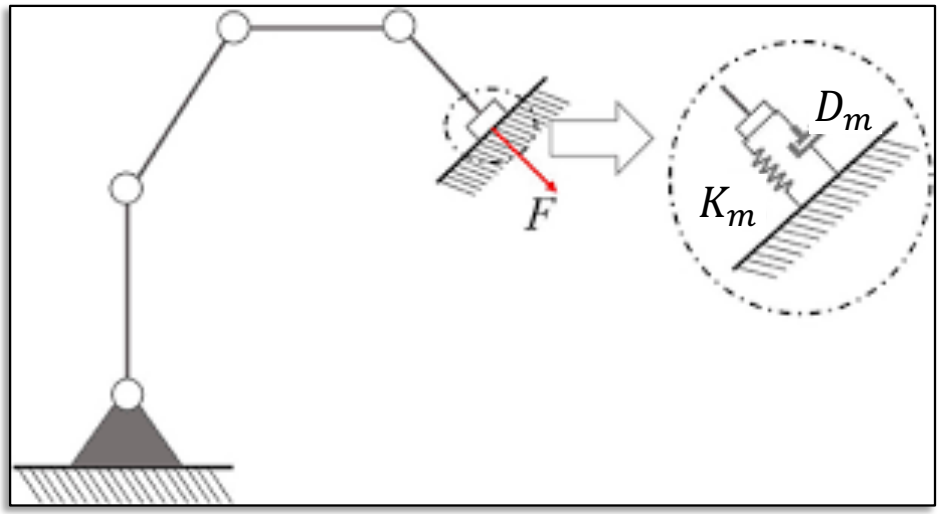
1. ***Linearization input***

$$\boldsymbol{u} = J_A^T(\boldsymbol{q})[B_r(\boldsymbol{q})\boldsymbol{v} + C_r(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{r}} + \boldsymbol{g}_r(\boldsymbol{q}) - \boldsymbol{f}_r]$$

2. ***Virtual input***

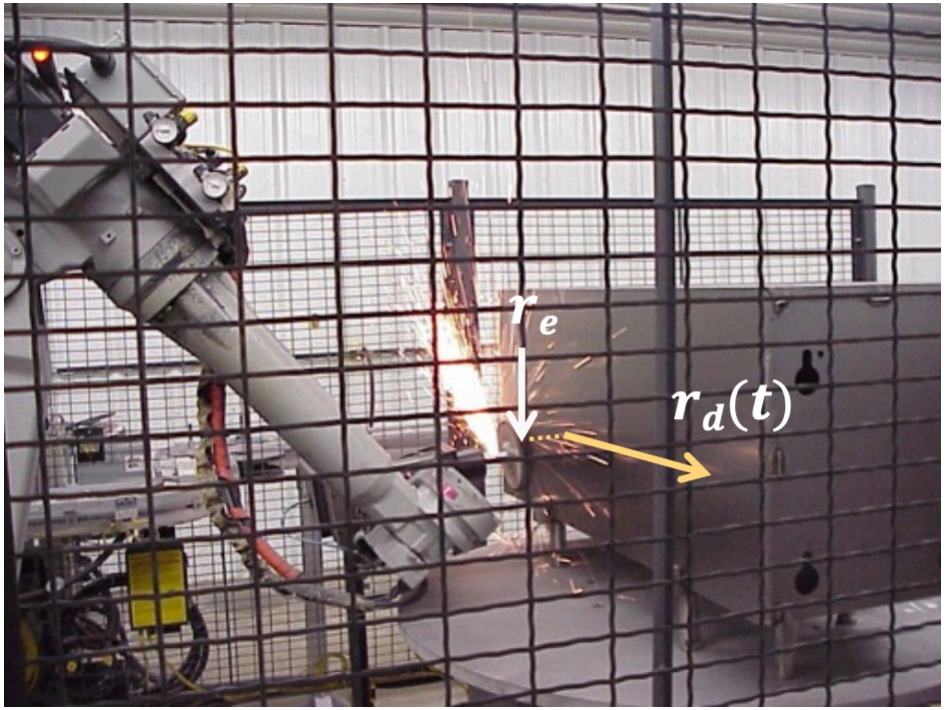
$$\boldsymbol{v} = \ddot{\boldsymbol{r}}_d + B_m^{-1}[D_m(\dot{\boldsymbol{r}}_d - \dot{\boldsymbol{r}}) + K_m(\boldsymbol{r}_d - \boldsymbol{r}) + \boldsymbol{f}_r]$$

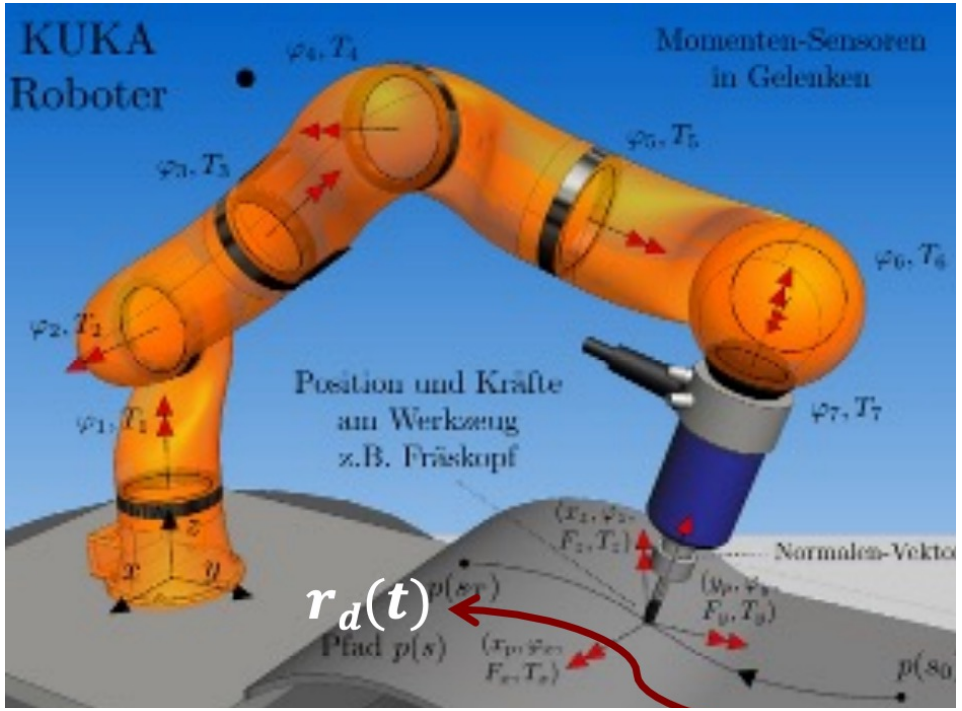
Desired inertia — **Desired damping** — **Desired stiffness**



Desired reference: examples

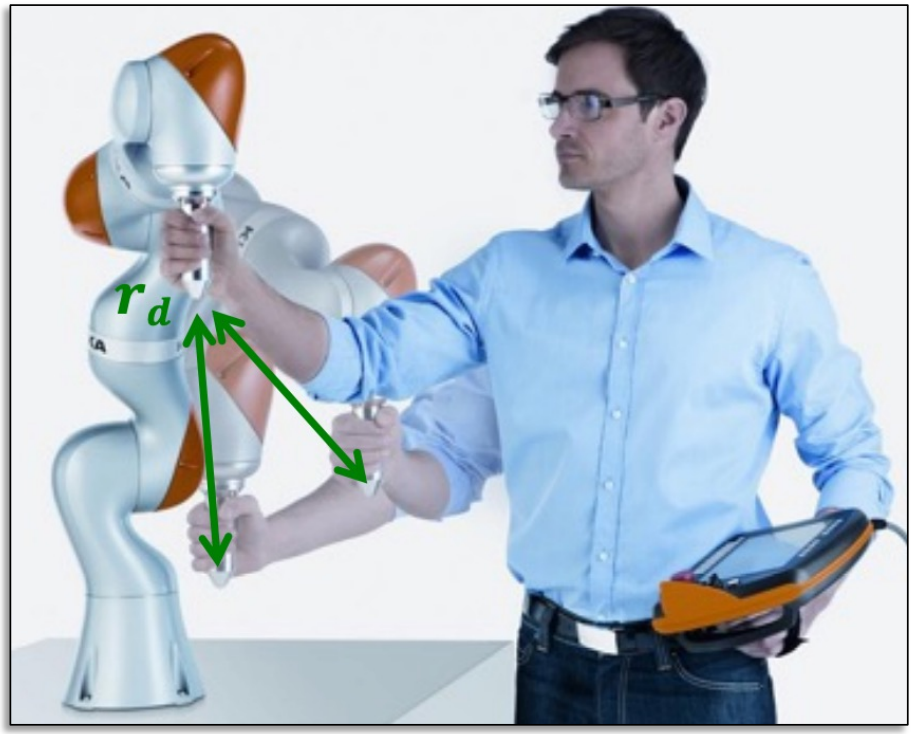
the desired motion may follow a surface (typically slightly inside the surface, to keep contact)

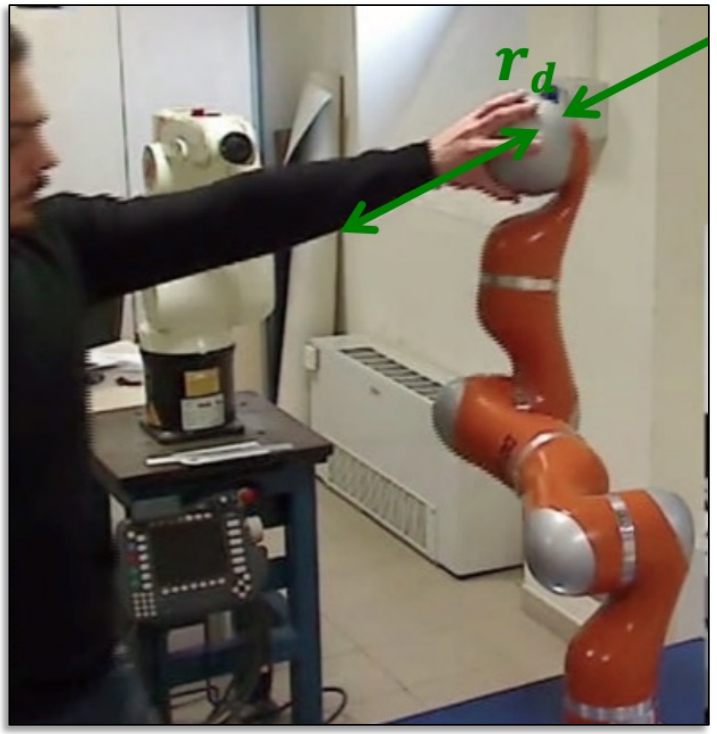




Desired reference

the desired motion can also be just a position ($\dot{r}_d = \ddot{r}_d = 0$) (compliance to external interactions, e.g. with a human)





Example: impedance vs. position control